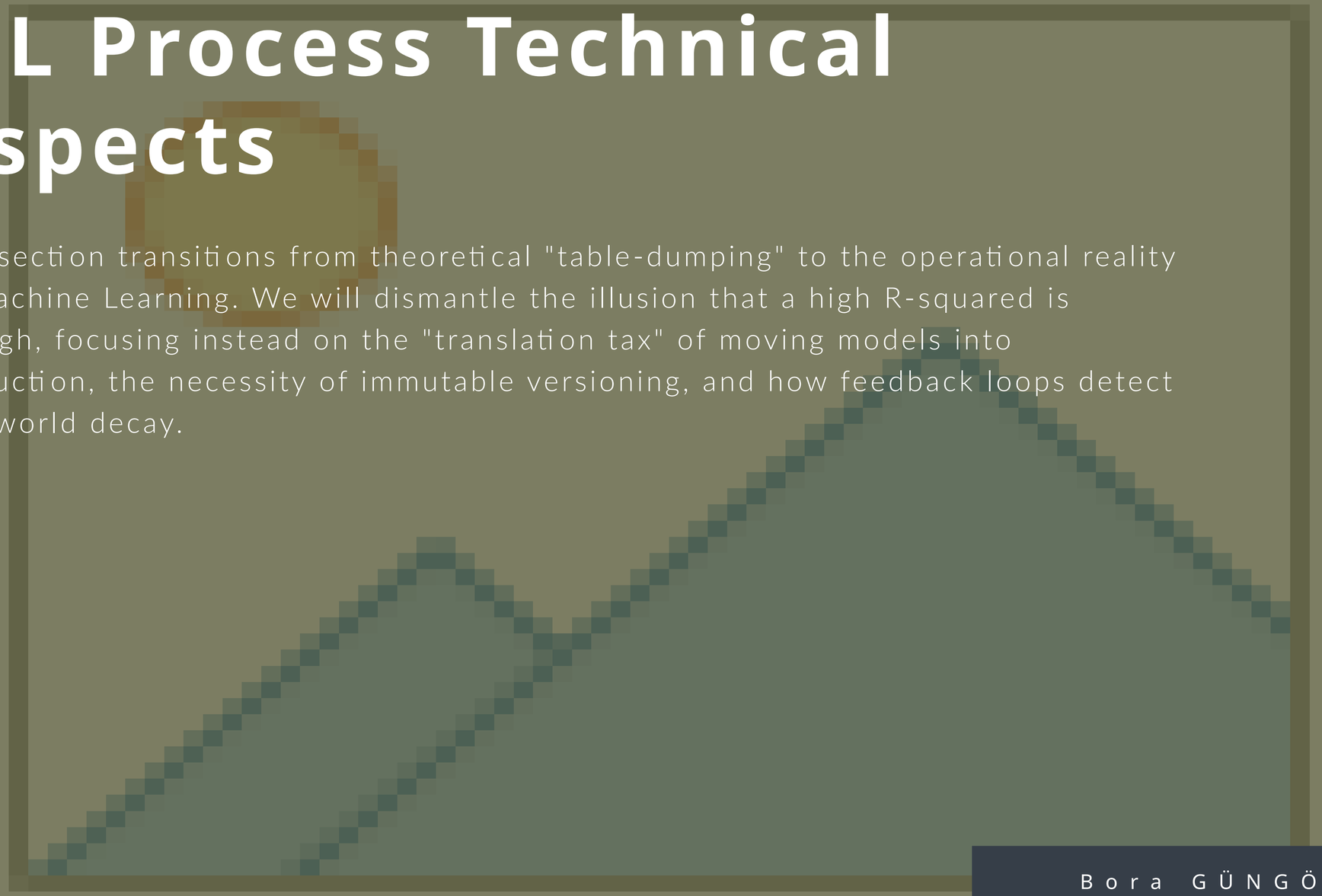
ML Process Technical Aspects

This section transitions from theoretical "table-dumping" to the operational reality of Machine Learning. We will dismantle the illusion that a high R-squared is enough, focusing instead on the "translation tax" of moving models into production, the necessity of immutable versioning, and how feedback loops detect real-world decay.

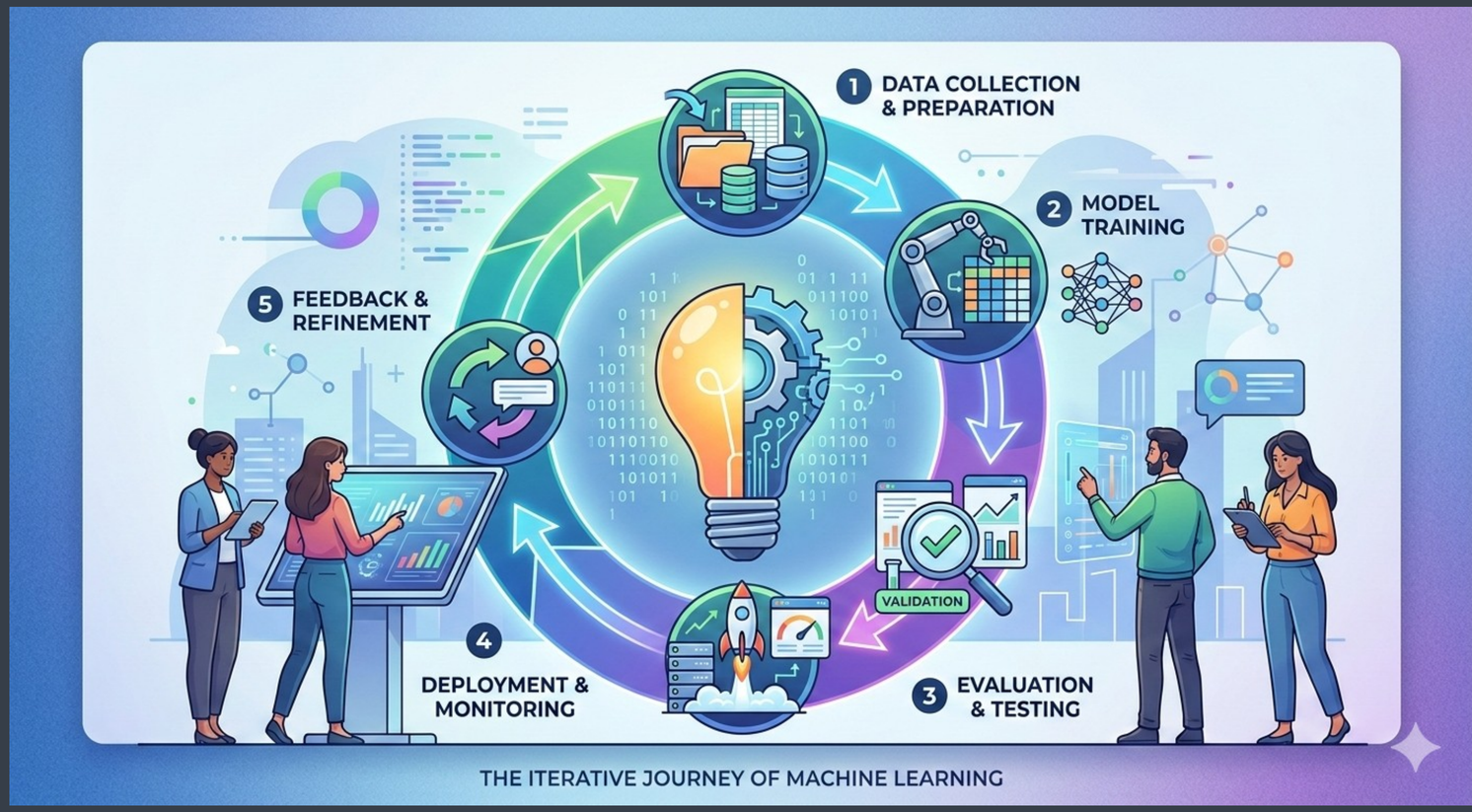


ECON484 Machine Learning

Bora GÜNGÖREN

The Lifecycle of a Model Artifact

In the previous session, we defined ML as a lifecycle (within an organization). In reality, **this lifecycle must run as a (software intensive) system** to be able to perform and contribute to organizational goals.

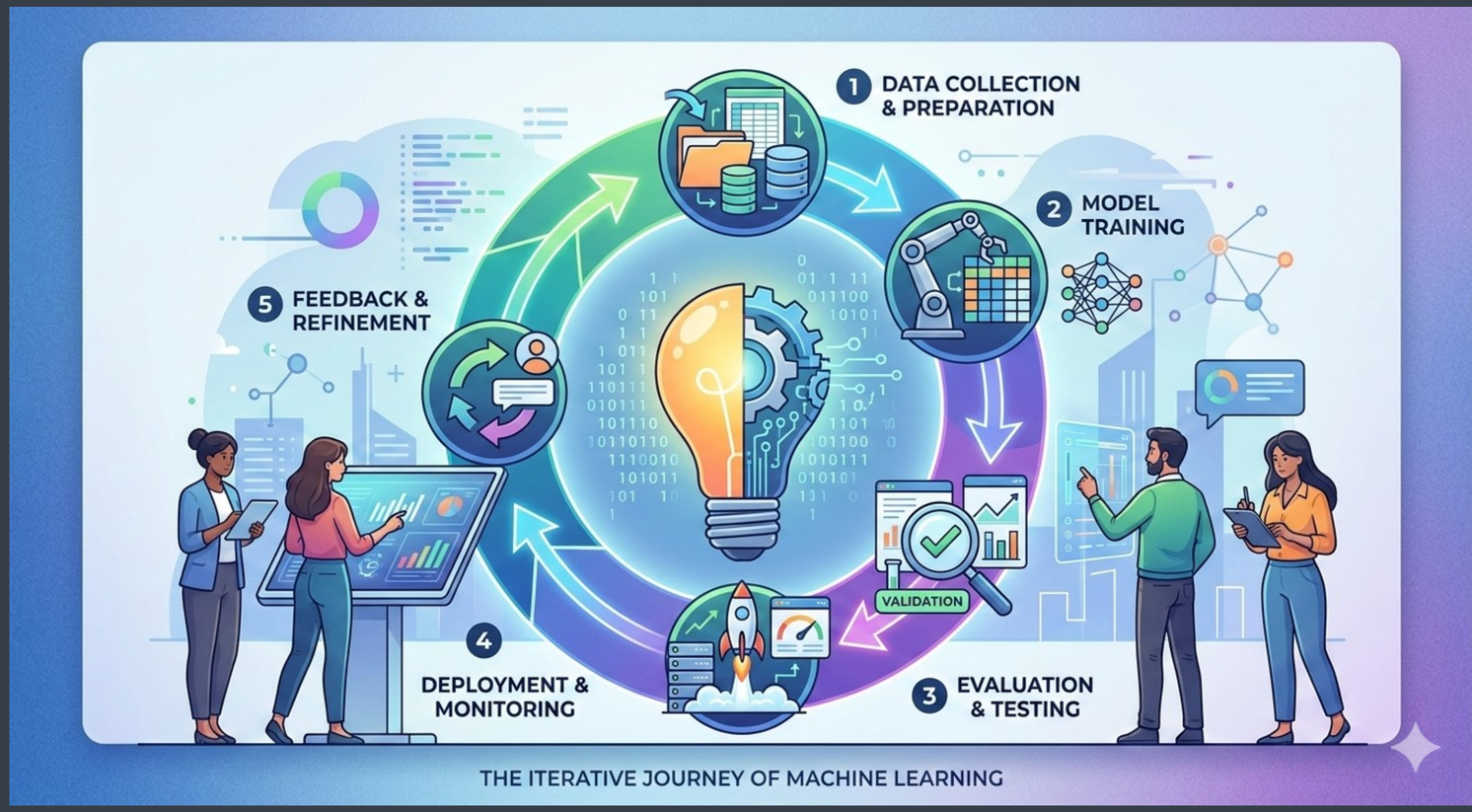


The Lifecycle of a Model Artifact

Every system has technical aspects.
- reproducibility
- versioning
- pipelines
- monitoring

Today we will discuss those. Some of them are simple, some are advanced.

You are not required to implement all of them in your course work. But you will **implement some lightweight practices**.

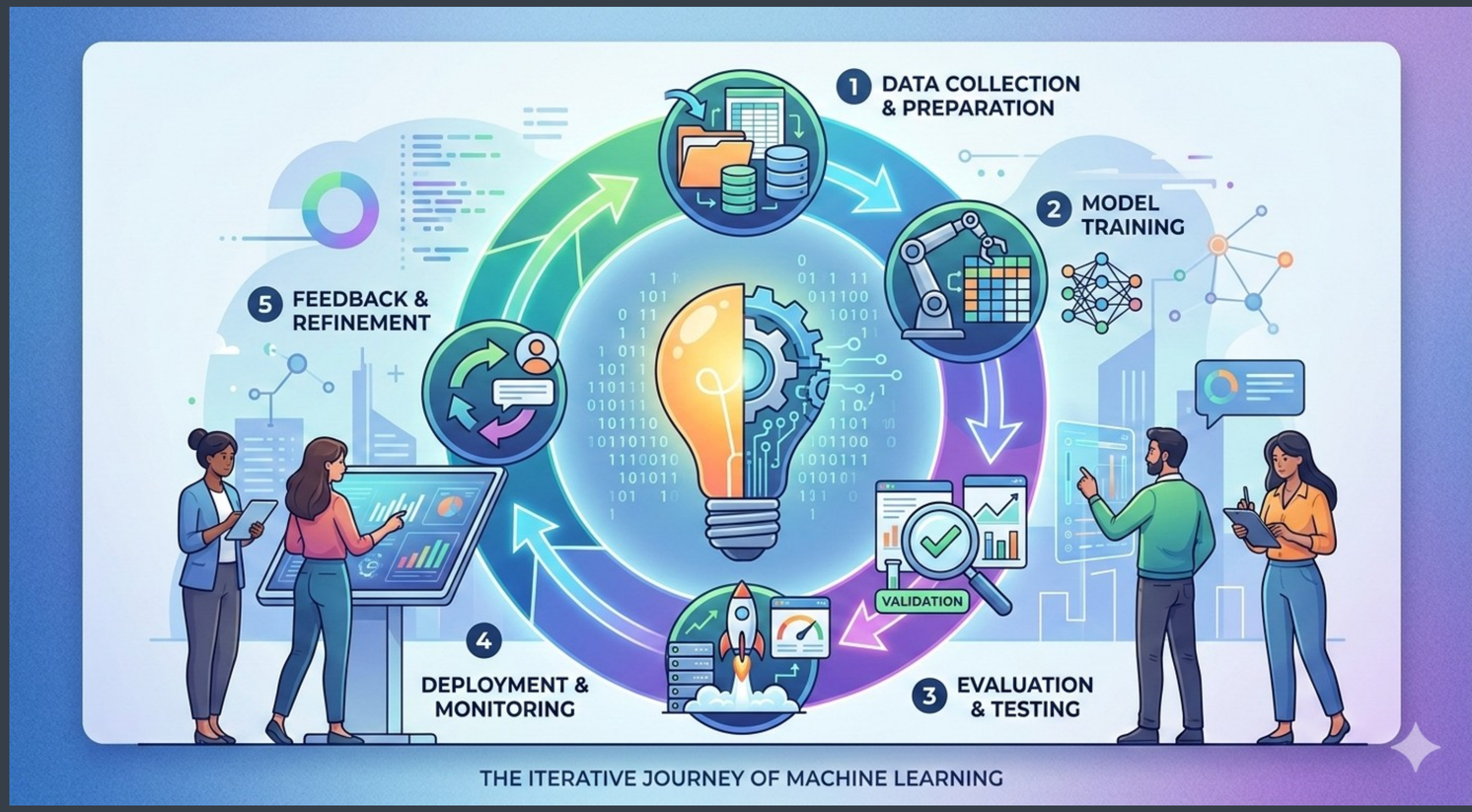


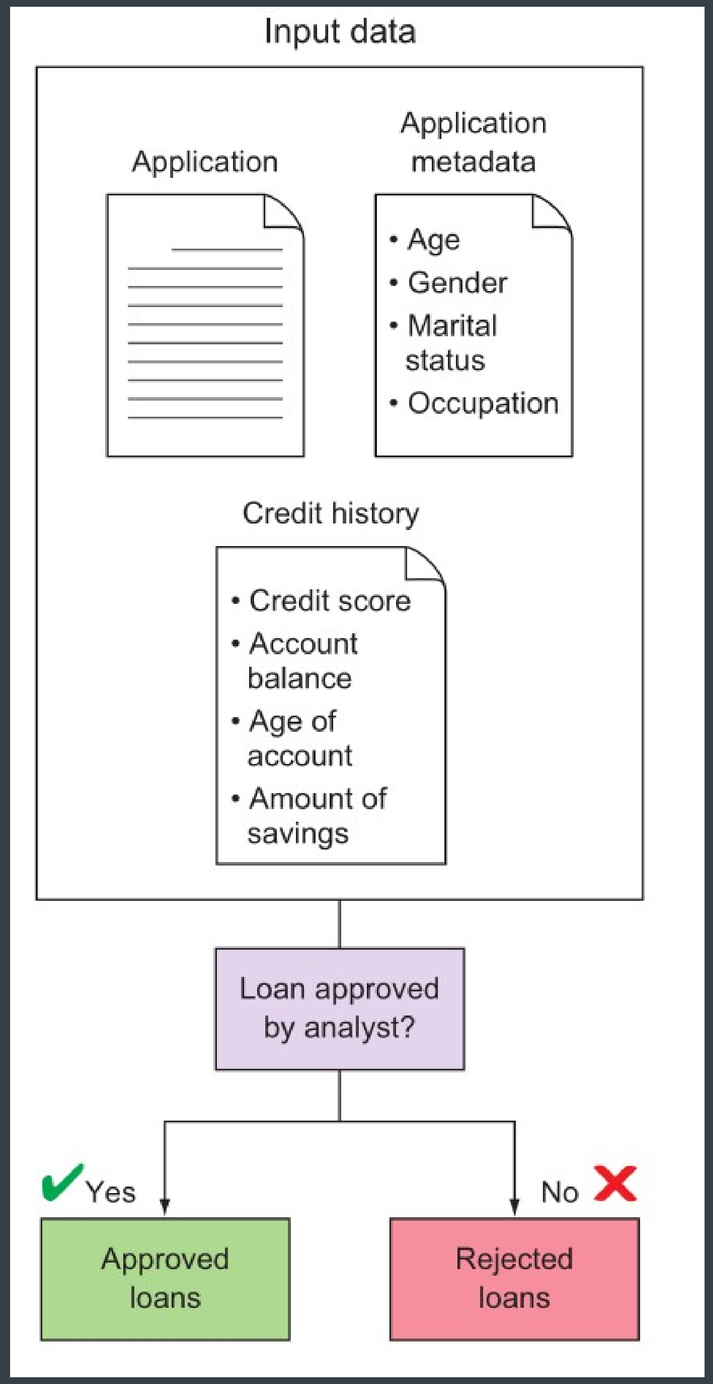
The Lifecycle of a Model Artifact

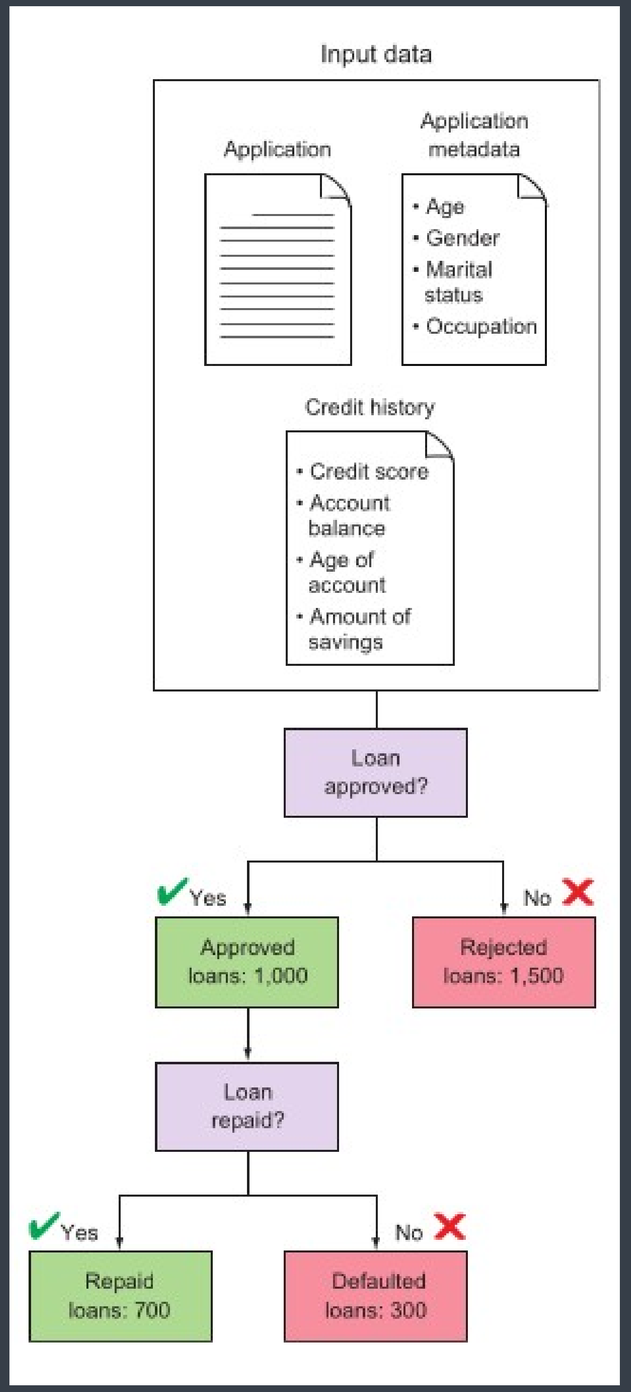
Transitioning from "Results" (SPSS output) to "Artifacts" (a serialized file) is the reality.

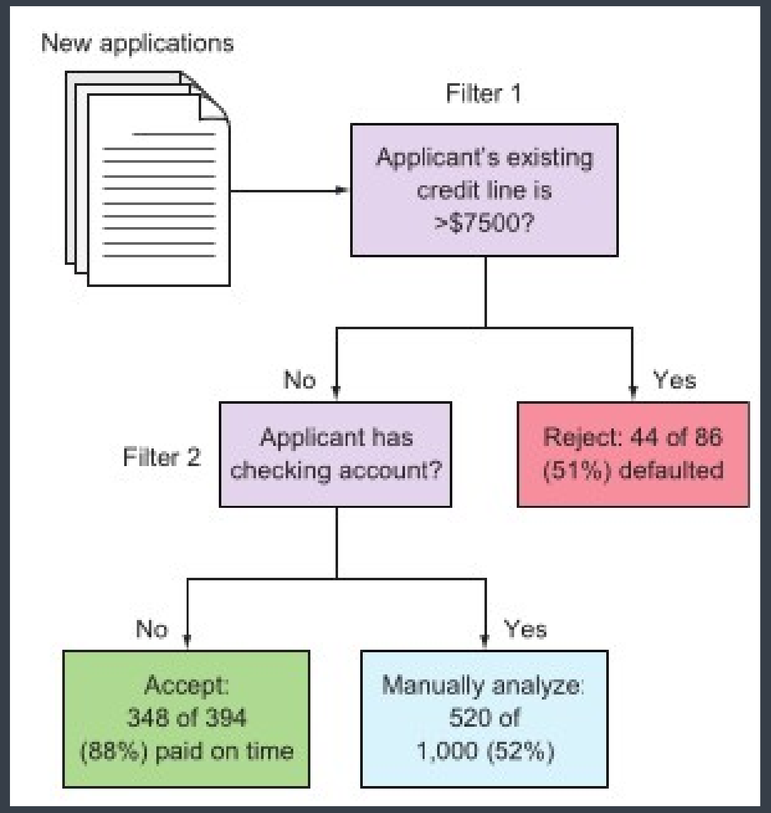
A model is a "frozen" snapshot of math and logic. Difference exists between Descriptive (explaining what happened) vs. Functional (predicting what will happen) in how it’s developed and how it’s deployed.

If an SPSS table is a photograph of a car, a Model Artifact is the actual engine.




Input data
Application
Application metadata
• Age
• Gender
• Marital status
• Occupation
Credit history
• Credit score
• Account balance
• Age of account
• Amount of savings
Loan approved by analyst?
Yes
No
Approved loans
Rejected loans

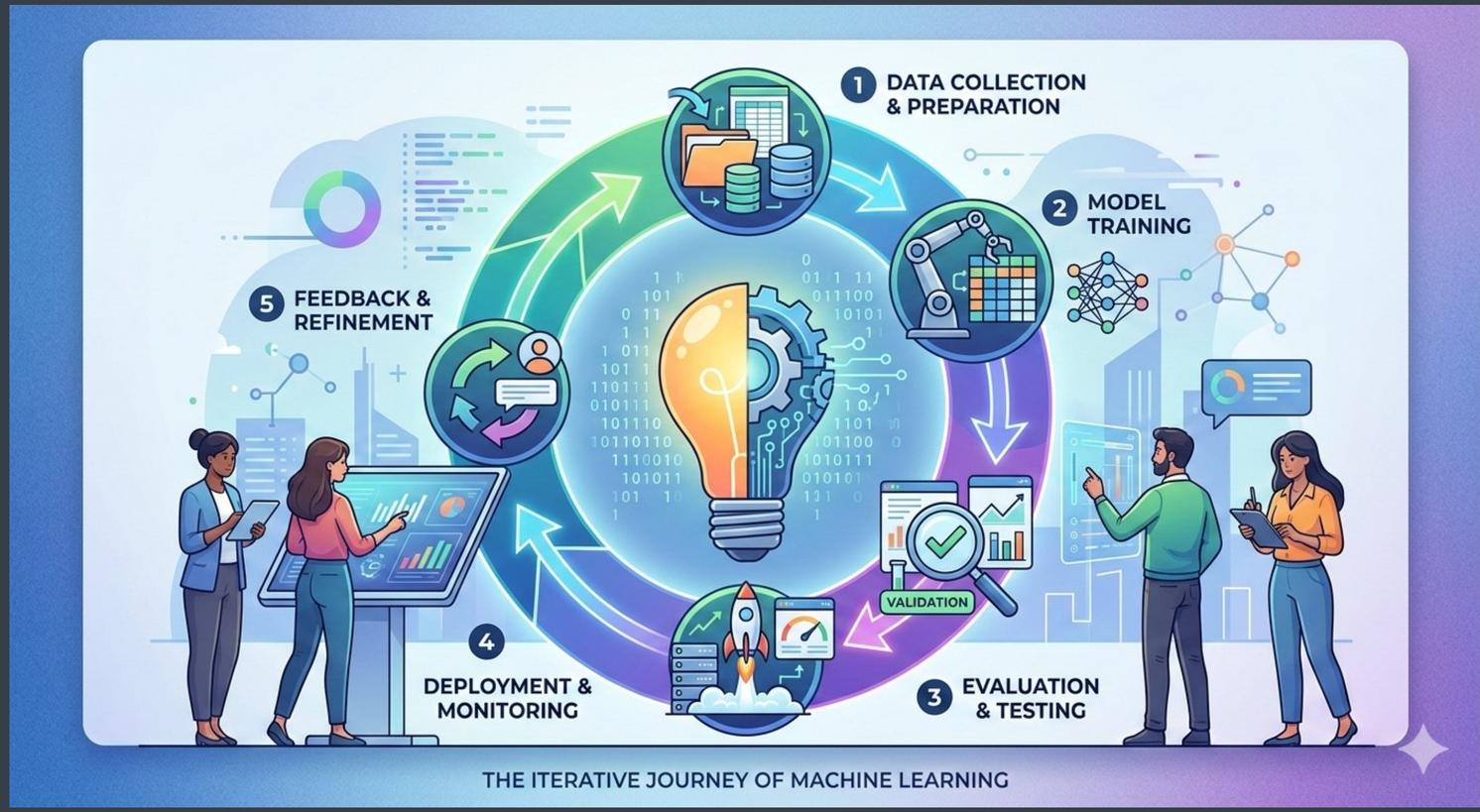

Input data
Application
Application metadata
• Age
• Gender
• Marital status
• Occupation
Credit history
• Credit score
• Account balance
• Age of account
• Amount of savings
Loan approved?
Yes
No
Approved loans: 1,000
Rejected loans: 1,500
Loan repaid?
Yes
No
Repaid loans: 700
Defaulted loans: 300

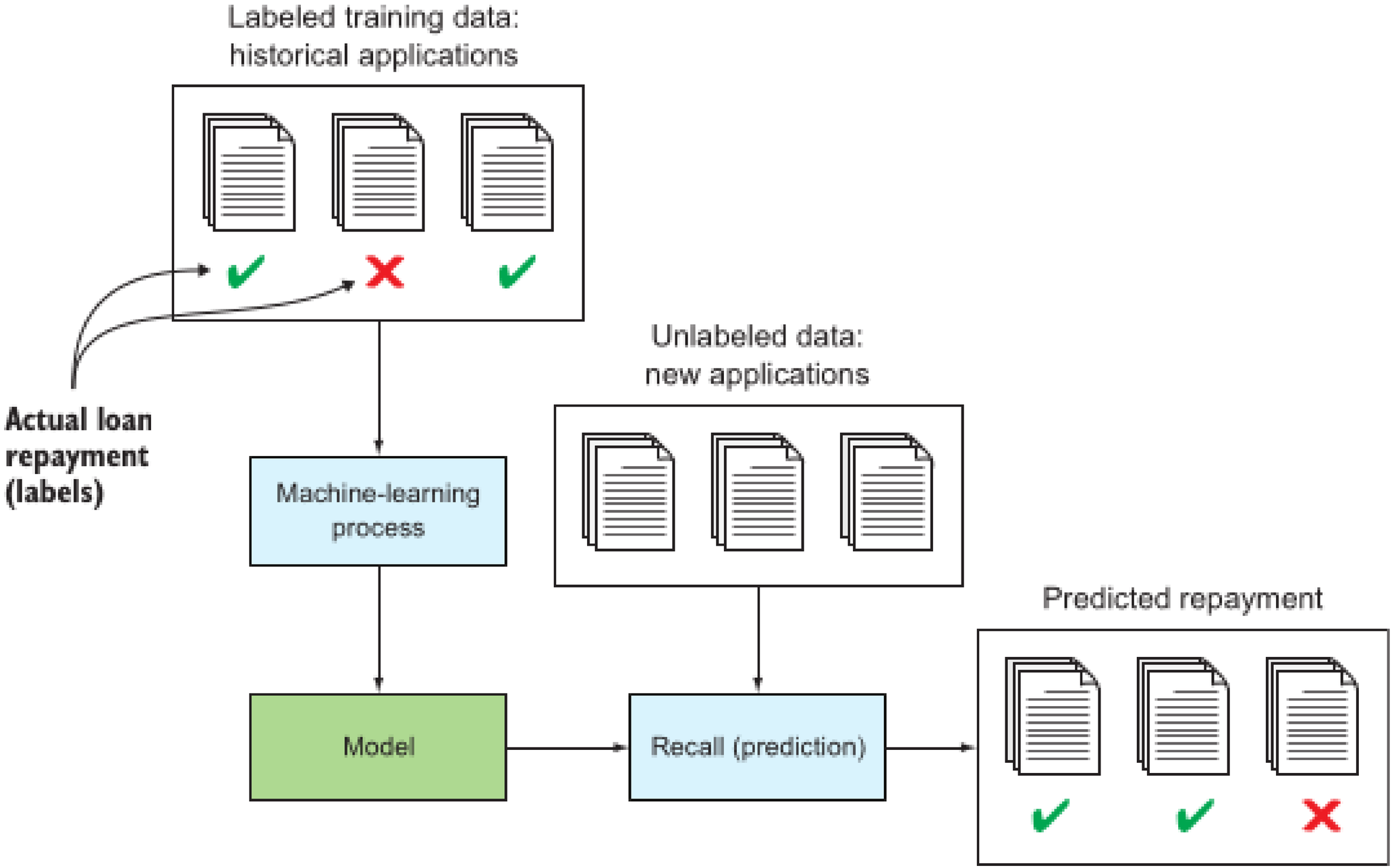

New applications
Filter 1
Applicant's existing credit line is >$7500?
No
Yes
Filter 2
Applicant has checking account?
Reject: 44 of 86 (51%) defaulted
No
Yes
Accept: 348 of 394 (88%) paid on time
Manually analyze: 520 of 1,000 (52%)

The Lifecycle of a Model Artifact

The friction of moving a model from the research environment to a production application is significant.

- "It worked on my laptop" vs. "It fails on the server."
- Versions of libraries (Python vs. C++/Rust) must match perfectly.
- Time spent re-coding feature logic in a different language is significant.



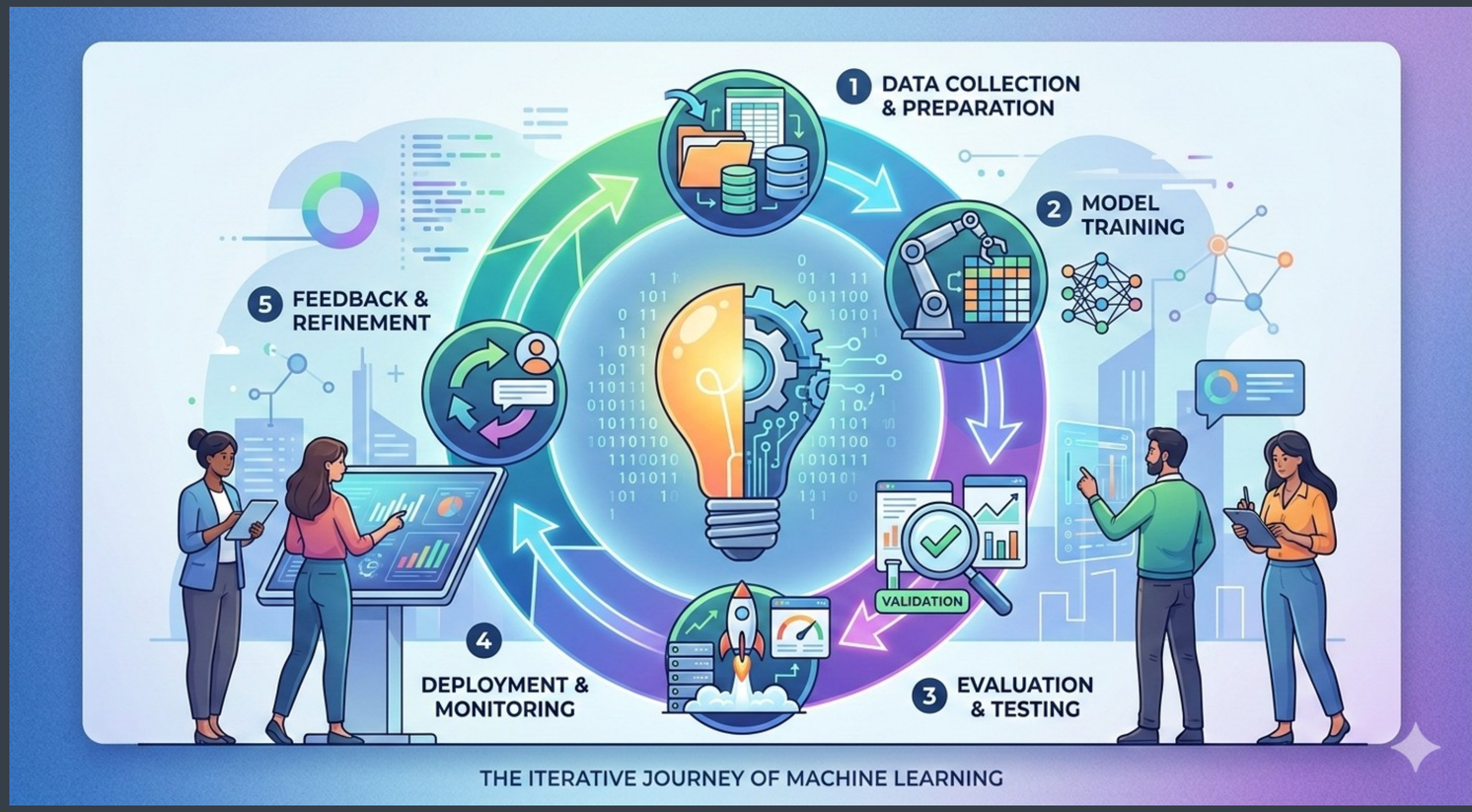

Labeled training data:
historical applications
Actual loan
repayment
(labels)
Machine-learning
process
Unlabeled data:
new applications
Model
Recall (prediction)
Predicted repayment

The Lifecycle of a Model Artifact

You cannot version the model alone; you must version the entire "recipe."

Data Version + Code Version + Hyperparameters = Specific Model Artifact. "Final_Model_v2_new.pkl" is a recipe for disaster.

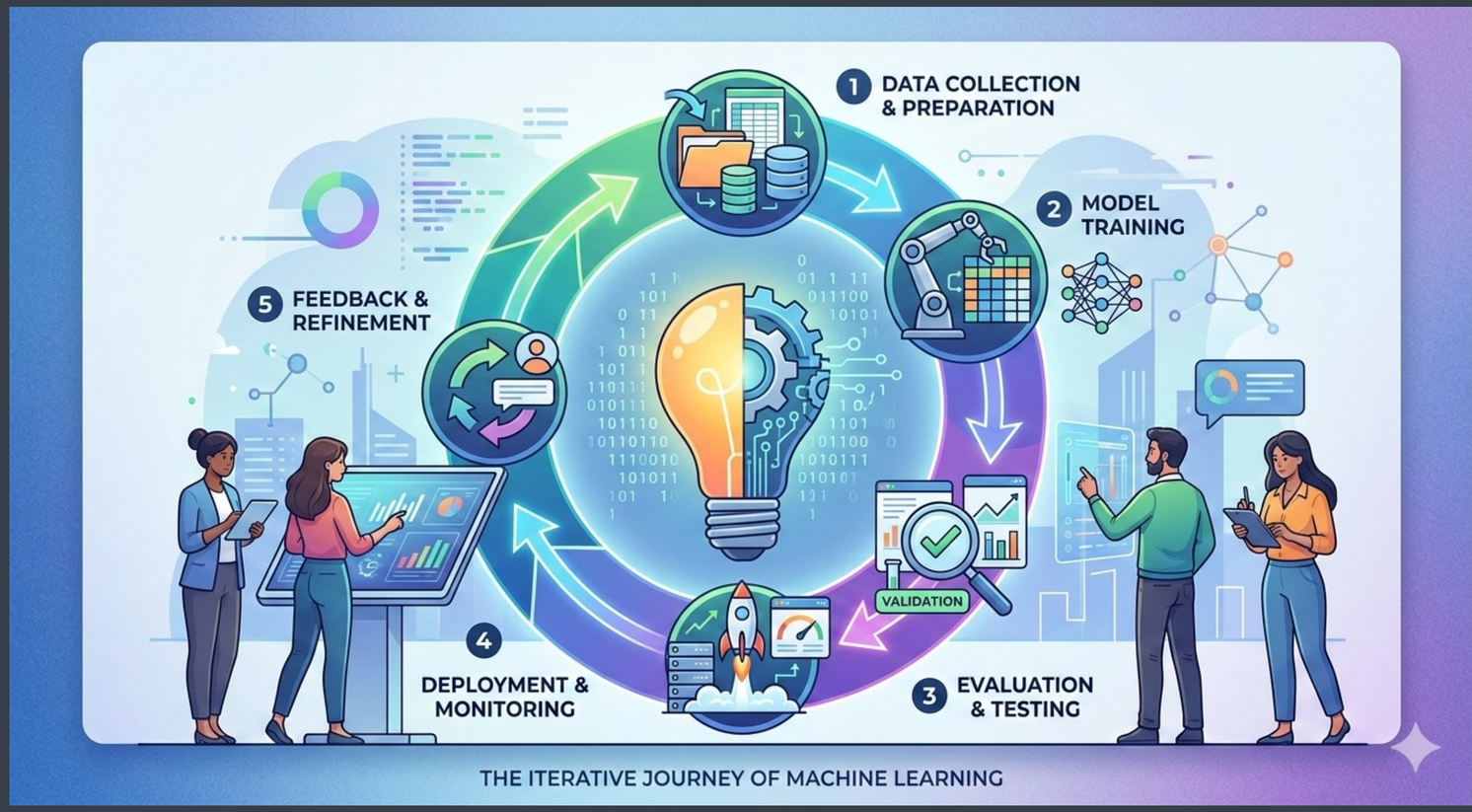
Experiment Tracking: Knowing exactly which data "birthed" which model. **(Core combined skill for this and next session)**.



The Lifecycle of a Model Artifact

Validation: Moving from statistical significance to operational reliability.
- We never test on the same data we used to "see" the patterns.
- Performance Metrics vs. Business Metrics
- Stress Testing: What happens when the model receives "junk" or missing data?

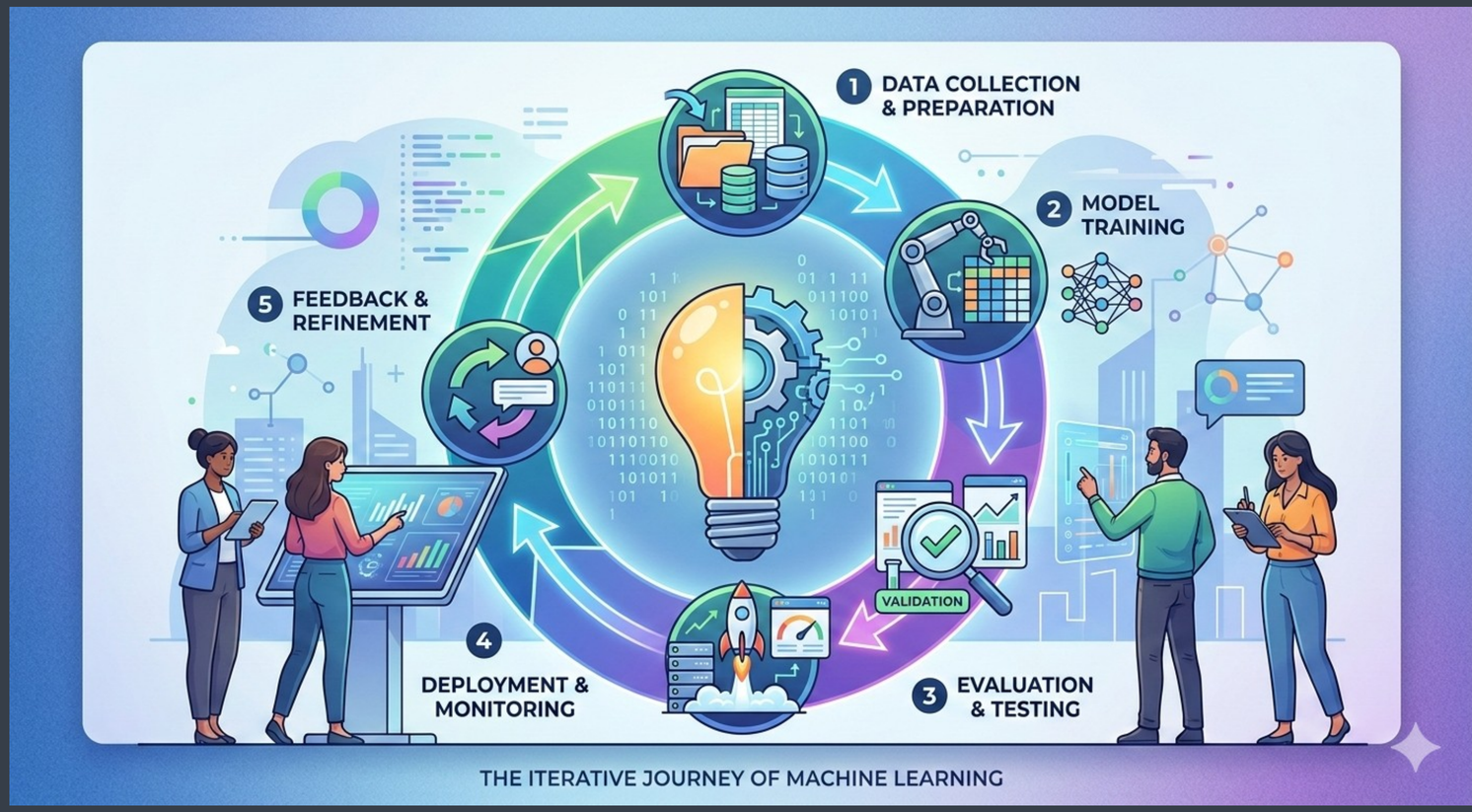
Validation is the next core combined skill you'll gain.



The Lifecycle of a Model Artifact

Living Artifacts: Models are not "set and forget." They begin to die the moment they are deployed.

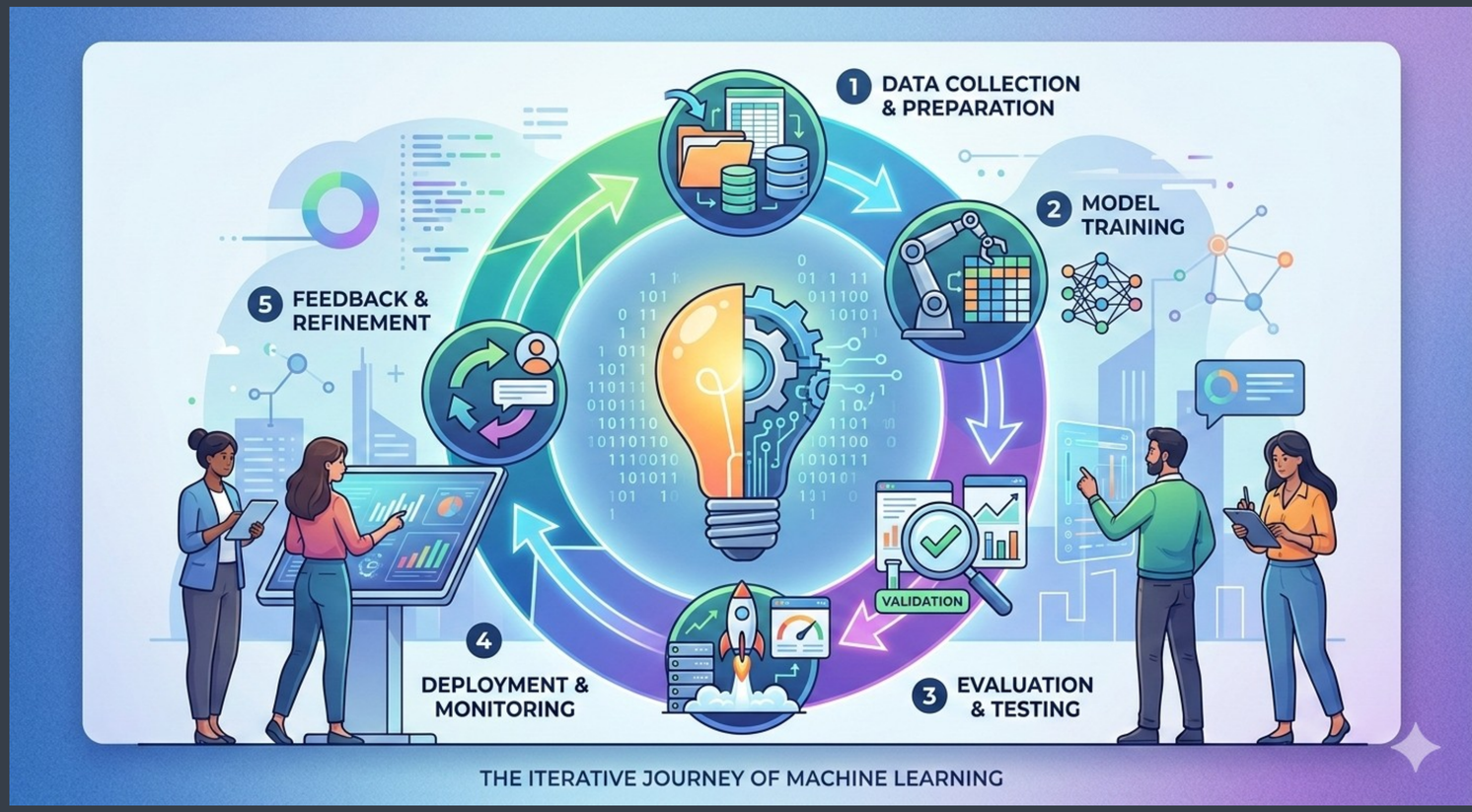
- Model Rot: Why last year’s consumer behavior model fails today.
- Feedback Loop: How the application sends real-world results back to the developer.
- Retraining Triggers: When is it time to "kill" the old artifact and deploy a new one?



The Reproducibility Crisis

In science and data analysis, if you cannot reproduce a result, the result is effectively useless. Hidden state makes your work non-reproducible.

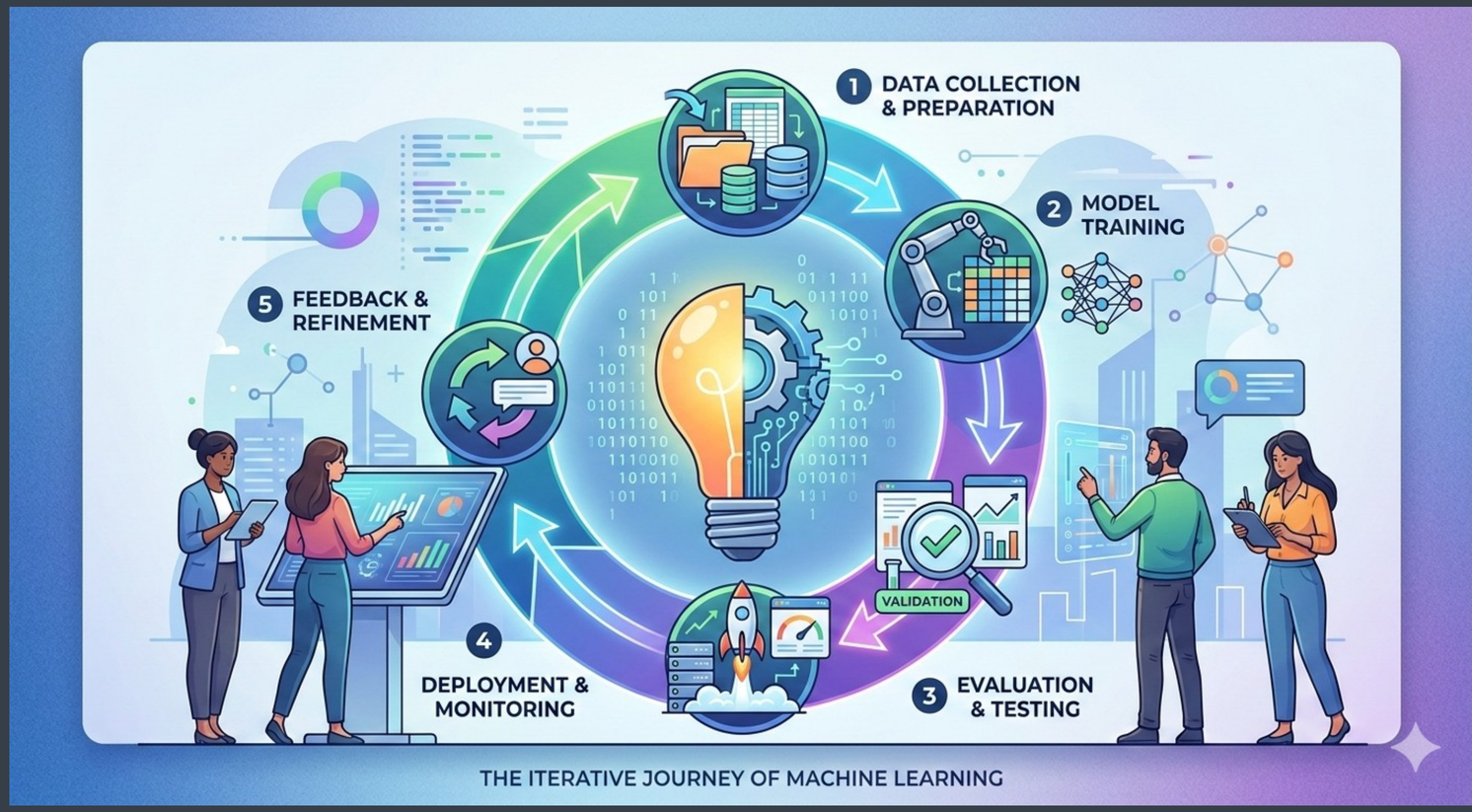
- Your reality: Your tools remember variable x from a cell you deleted two hours ago.
- True reality: The notebook file does not contain the instruction to create x.
- Result: "It works on my machine" syndrome. No one else can recreate your work.



The Reproducibility Crisis

How does this happen?

"It Worked Once" Fallacy: Statistical significance in a vacuum is not science; it’s a fluke.
- The "Excel/SPSS Trap": Manually changing a cell or clicking a filter without logging it.
- Definition of Reproducibility: Same Data + Same Code = Same Artifact.



The Reproducibility Crisis

The danger of "Hidden State": Why standalone models (ie. Jupyter notebooks) that are run out of order create false realities.
- Creates a disconnect between what the code says (the visible text in the cells) and what the computer knows (the variables stored in memory).
- In a standard script: You run the file from top to bottom. Line 10 cannot run before Line 5. The state is strictly tied to the linear flow of the text. In a Jupyter Notebook: You can run Cell 1, then Cell 5, then go back and run Cell 2. The Kernel simply executes whatever you tell it, whenever you tell it. The "state" (variables, imports, function definitions) persists in memory regardless of where the code sits visually on the page.
- Most common error: "Deleted Code." You work interactively with the data, and see good results at the end. Then you clean up your code, deleting some of the code. Your previous results are still visible on the UI. But the artifact has changed. You are fooled by your toolset. If you submit your code without re-running everything and checking the results again, you fail.

The danger here goes beyond simple bugs; it affects scientific integrity and operational reliability.

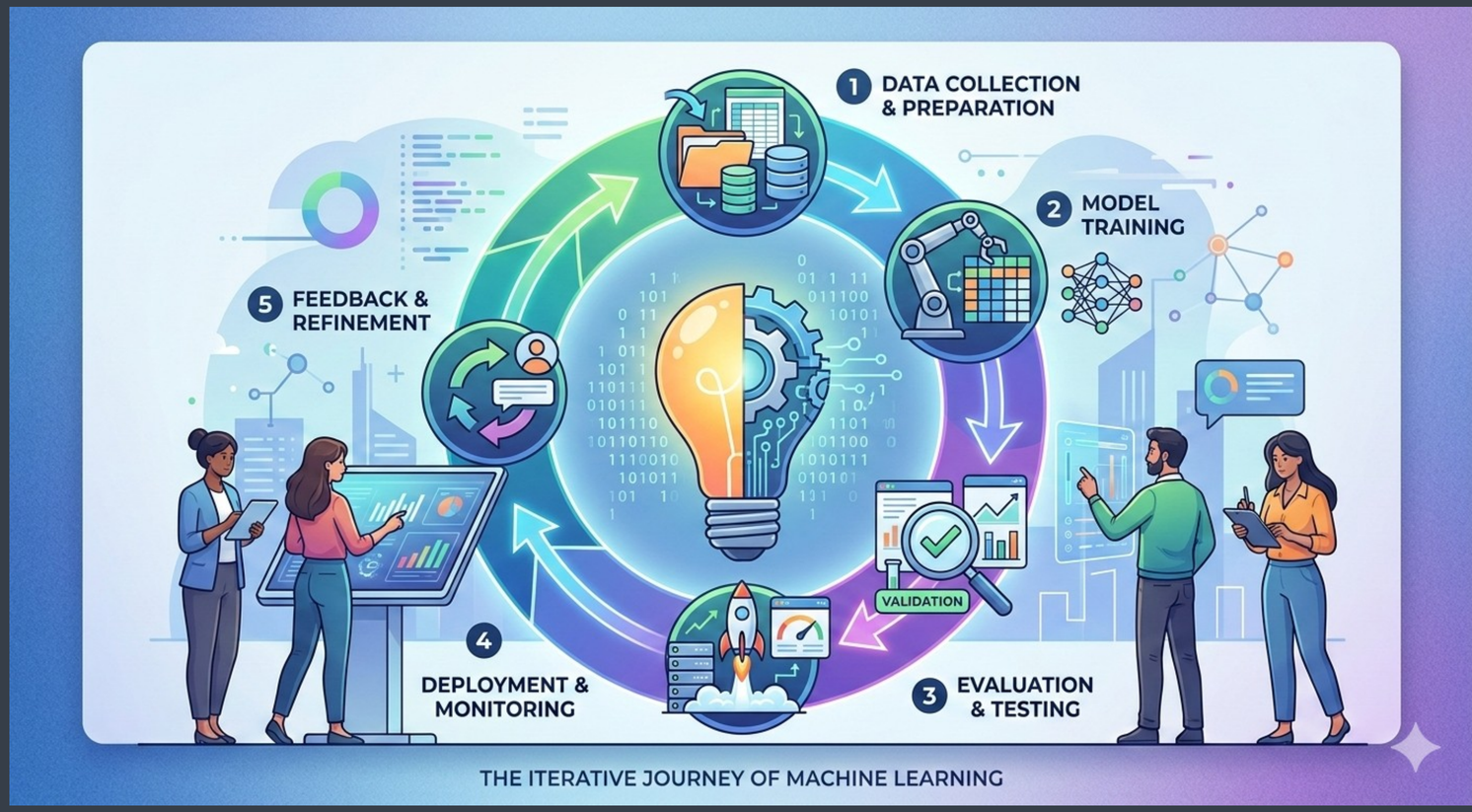
The Reproducibility Crisis

To prevent hidden state from creating false realities, adopt these habits:

- The "Restart and Run All" Ritual: Before sharing, committing, or trusting a notebook or project, always restart the kernel and run all cells from top to bottom. This forces the computer to forget the "hidden state" and try to recreate reality based strictly on the code written.
- **Notebooks are for Exploration, Scripts are for Production**: Notebooks are excellent for experimenting, visualizing, and telling a story. Once the logic is solid, move the code into a .py script. Scripts force linear execution and eliminate the ability to run code out of order.
- Treat the Notebook as Read-Only for State: Try to write code as if the cells above are the only things that determine the current cell's outcome. Never rely on a variable that isn't defined in a cell above the current one.

The Reproducibility Crisis

Data Versioning is Crucial: Treat data as a moving target. Always version.
- The problem of "Data Leakage": Accidentally training on information from the future.
- Point-in-time recovery: Can you revert your training database to exactly how it looked last Tuesday at 10:00 AM?
- "Dataset_Final_v2.csv" is a systemic risk.



The Reproducibility Crisis

In a production environment, data is alive. It breathes and changes.

- Monday: Your database has 1,000 records. You train a model. Accuracy is 85%.
- Wednesday: A data engineer fixes a bug in the ETL pipeline. Suddenly, 50 of those records have different values. The database now has 1,000 records, but the data inside them has changed silently.
- Friday: You try to retrain the model. The code hasn't changed, but the accuracy is now 80%.

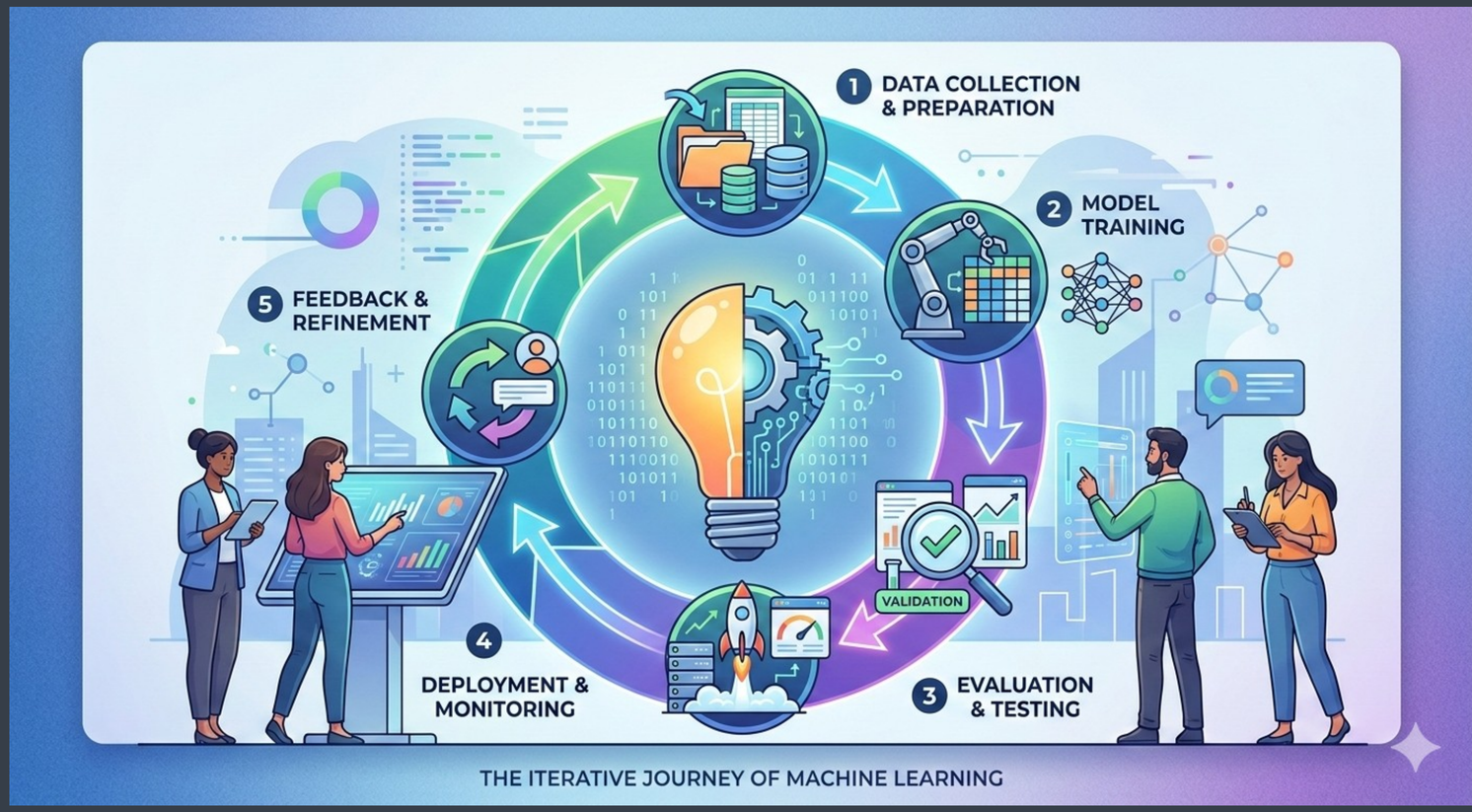
If you assume data also has not changed, then “Data Version + Code Version + Hyperparameters = Specific Model Artifact” seems to fail.

Without data versioning, you have no idea what changed. You are shooting at a moving target while blindfolded. Data versioning means taking a snapshot of your data at a specific moment so you can "freeze" that target.

The Reproducibility Crisis

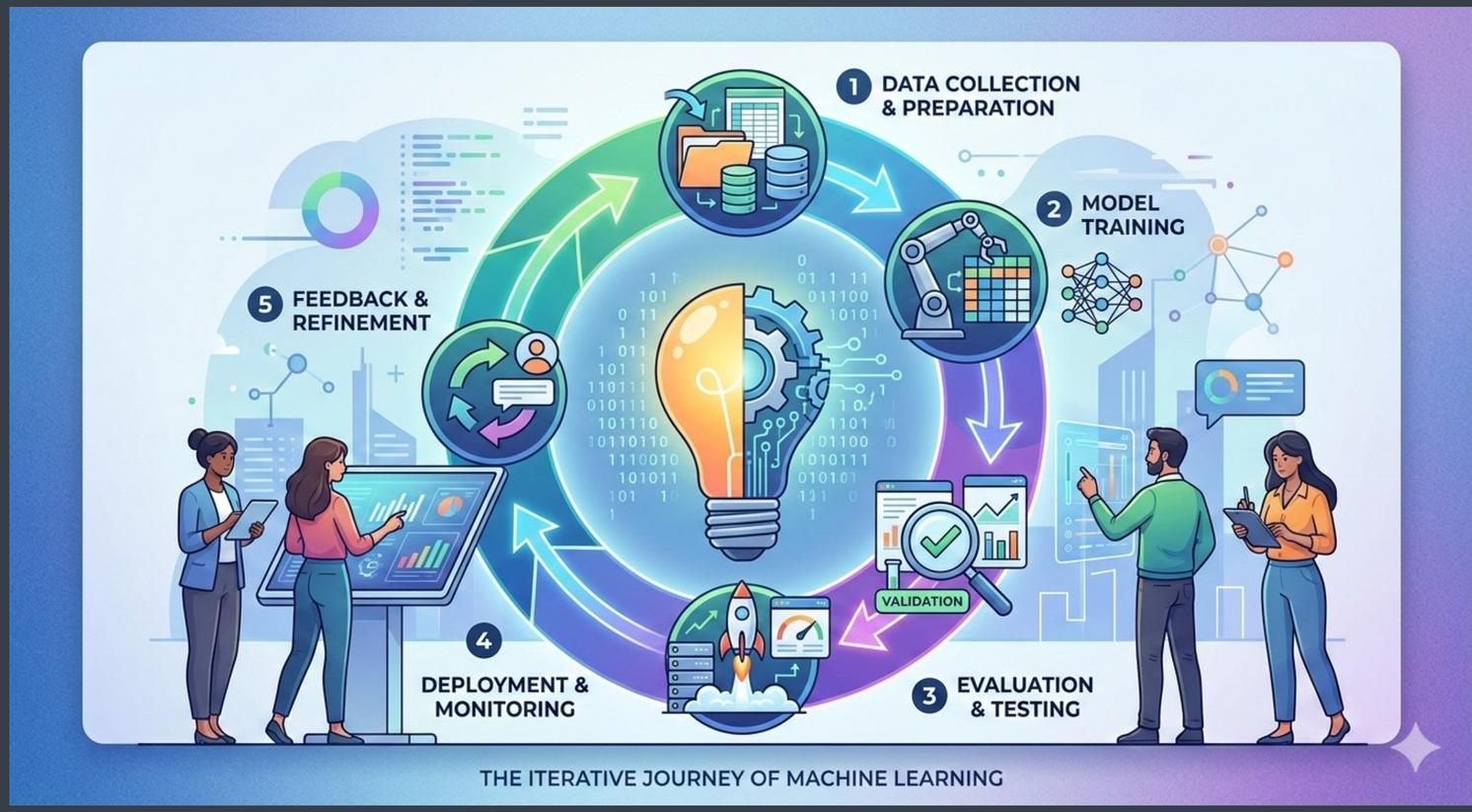
Software libraries are not neutral; they change the math.

- A version update in NumPy or Scikit-learn can change a rounding function and break a model.
- **Containers (Docker):** Freezing the operating system and libraries so the model "lives" in a bubble. (Advanced skill for this course level, but keep it in your learning path).
- Hardware Determinism: A model trained on a GPU might give slightly different results on a CPU.



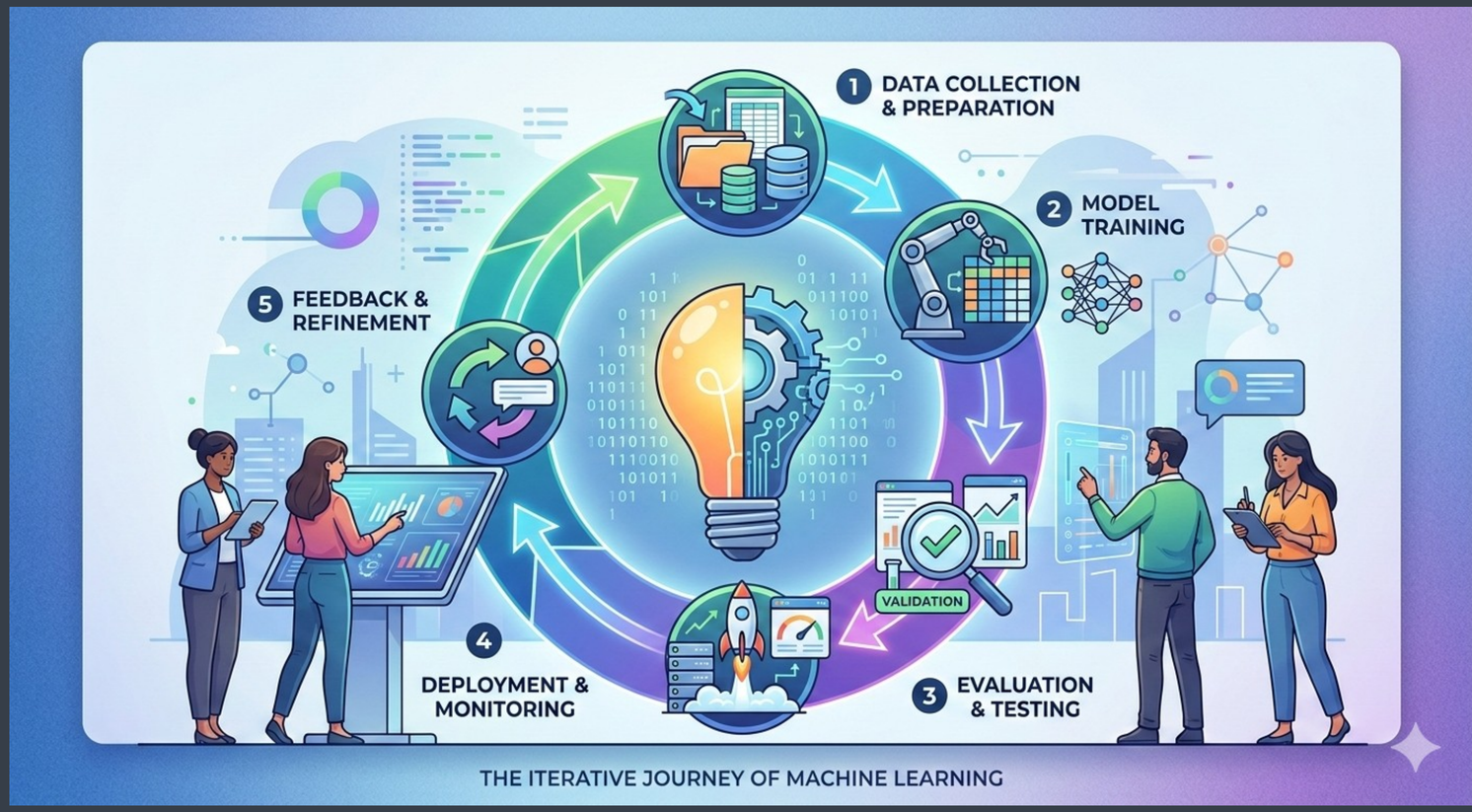
The Reproducibility Crisis

Randomness is a Variable: Computers are bad at being random, and that affects your "p-values."
- The "Random Seed": If you don't fix your seed, your train/test split changes every time.
- Stochastic Algorithms: Many ML models initialize with random weights.
- The "Lucky Seed" Scam: Warning against running a model 100 times and only reporting the best "lucky" result.



The Reproducibility Crisis

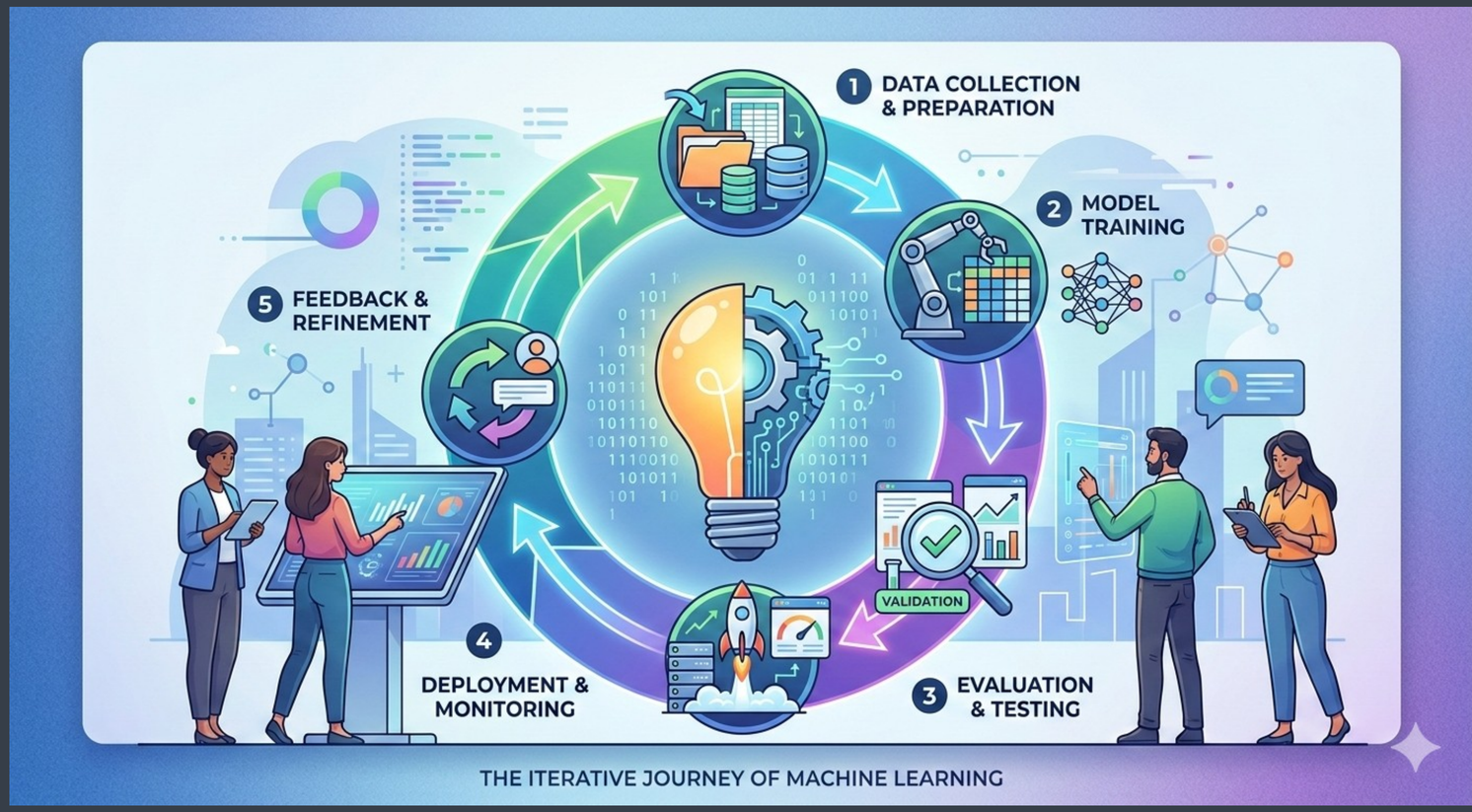
Experiment Ledger: It is a structured, traceable system used to record, track, and audit experiments — most commonly in data science, machine learning, and scientific R&D contexts.



The Reproducibility Crisis

As a system of record for experimentation, analogous to a financial ledger, but instead of transactions, it logs:
- Hypotheses
- Configurations (parameters, datasets, code versions)
- Execution details
- Results and metrics
- Conclusions / decisions

The goal is reproducibility, comparability, and governance.



The Reproducibility Crisis

Key Components of an Experiment Ledger

1. Experiment Metadata: Unique experiment ID, Author / owner, Timestamp, Objective or hypothesis
2. Inputs: Dataset version (critical for ML reproducibility), Feature engineering steps, Hyperparameters / configuration
3. Execution Context: Code version (e.g., Git commit hash), Environment (libraries, hardware, GPU/CPU), Pipeline definition
4. Outputs: Metrics (accuracy, loss, latency, etc.), Artifacts (models, logs, plots)
5. Lineage & Relationships: Parent/child experiments, Branching (e.g., hyperparameter sweeps), Comparison history

Lightweight Implementation:

- Spreadsheets for the Ledger itself
- Notebooks and scripts with structured logging
- Git + naming conventions

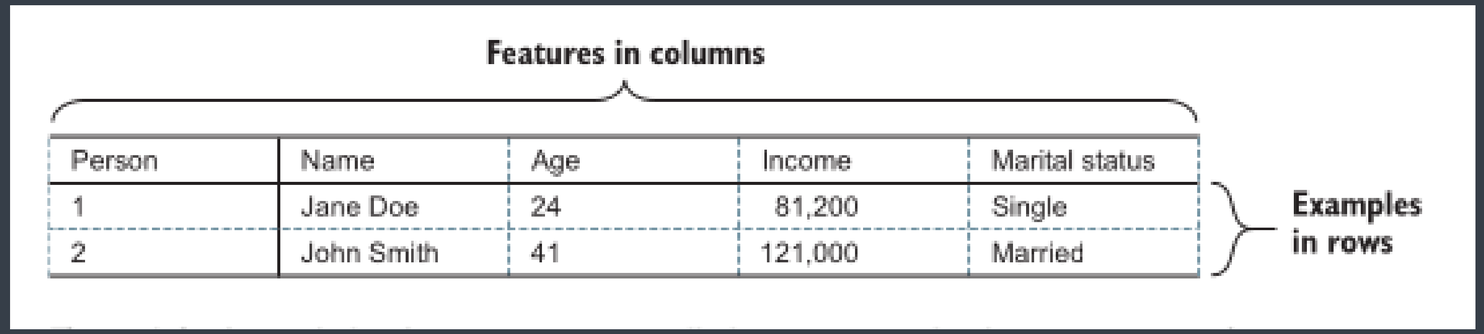
The Reproducibility Crisis

Advanced Implementation:

- Moving from manual notes to automated tools (MLflow, Weights & Biases).
- The Metadata: Logging the user, the date, the git commit hash, and the hardware specs.
- The Audit Trail: Being able to prove to a regulator (or a boss) exactly how a specific decision was reached by the model.

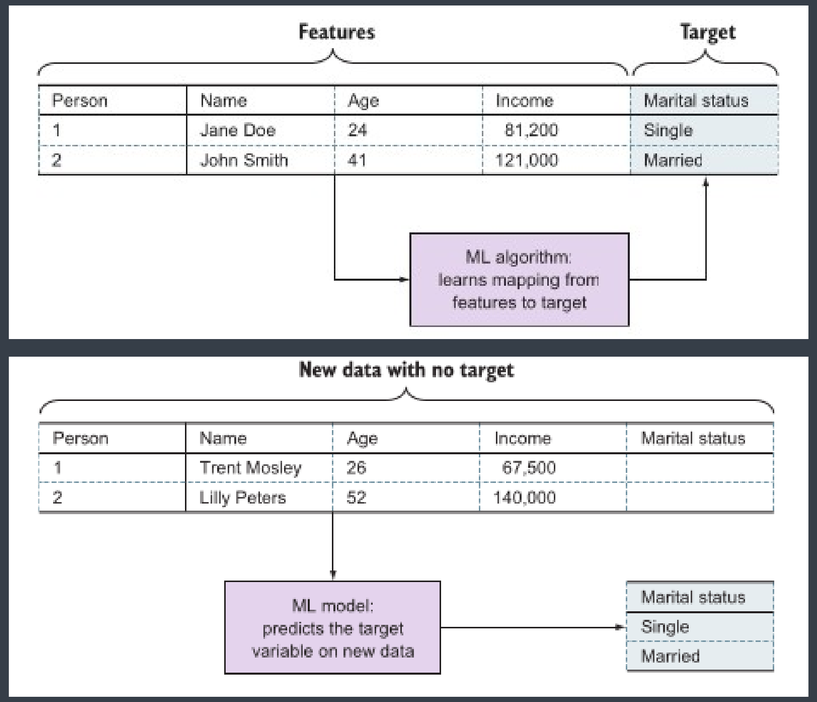
Feature Pipelines

In a tabular dataset, rows are called instances and columns represent features.



Feature Pipelines

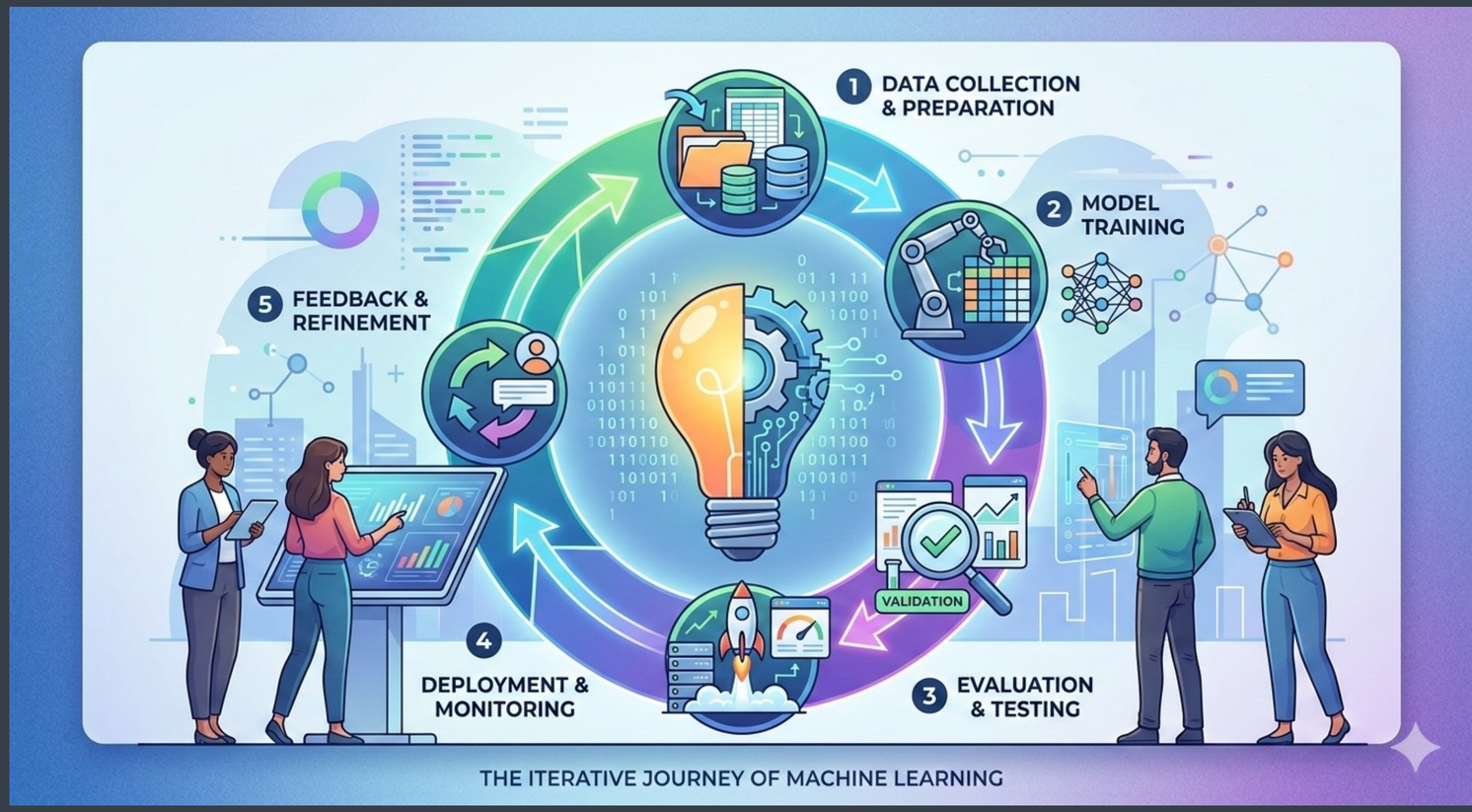
ML algorithms learn **mappings** from features to targets. Then we use models for prediction on new data.



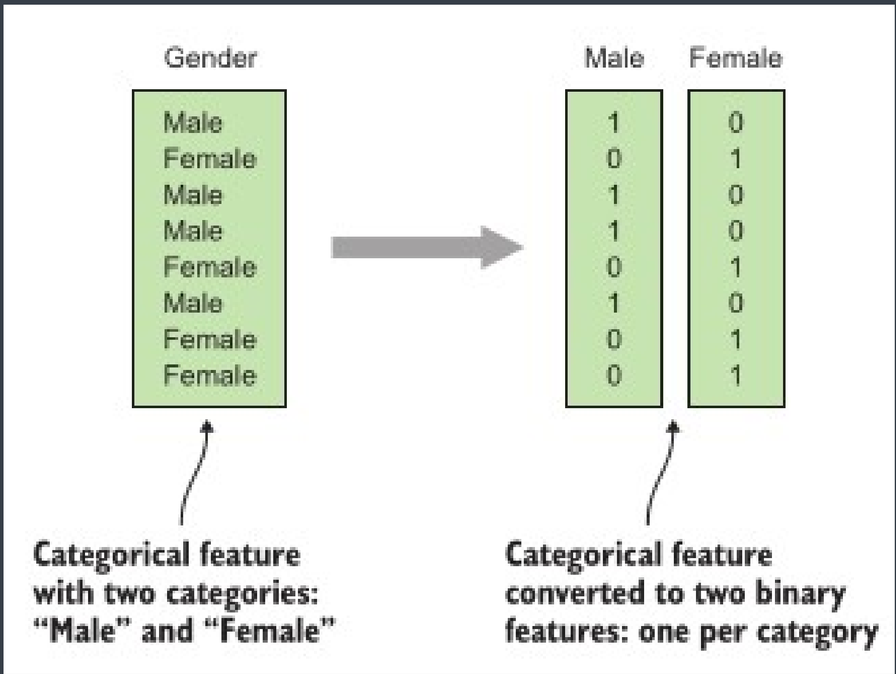
Feature Pipelines

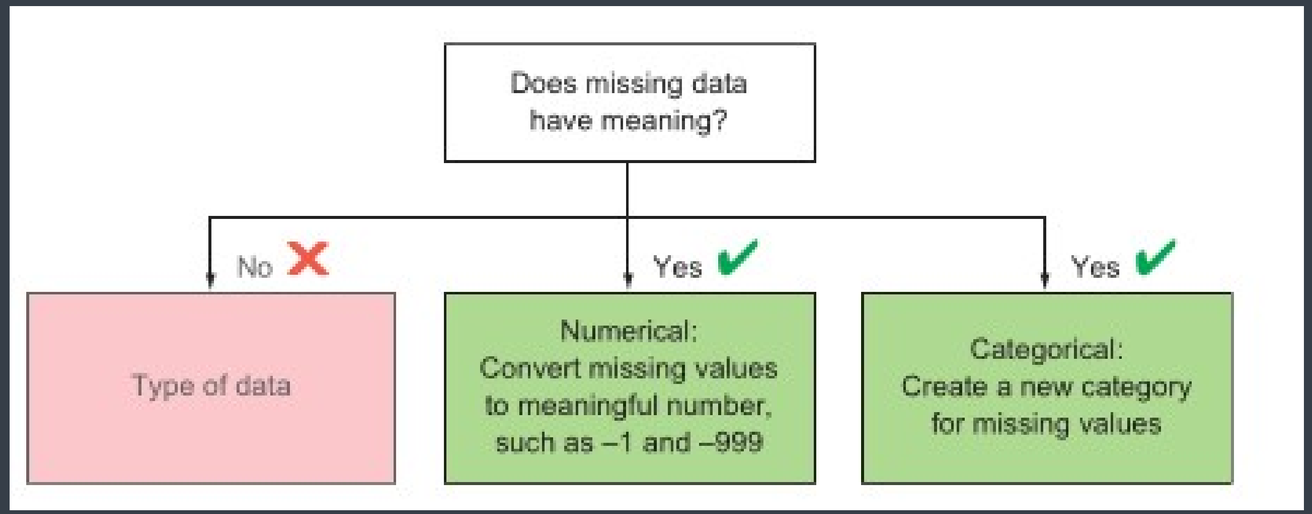
Feature engineering is the process of **transforming raw data into structured inputs** (“features”) that improve a model’s ability to learn patterns.

In practice
- Cleaning (handling missing/outliers)
- Encoding (categorical, numeric)
- Scaling/normalization
- Creating new features (aggregations, ratios, time-based signals)



Feature Pipelines



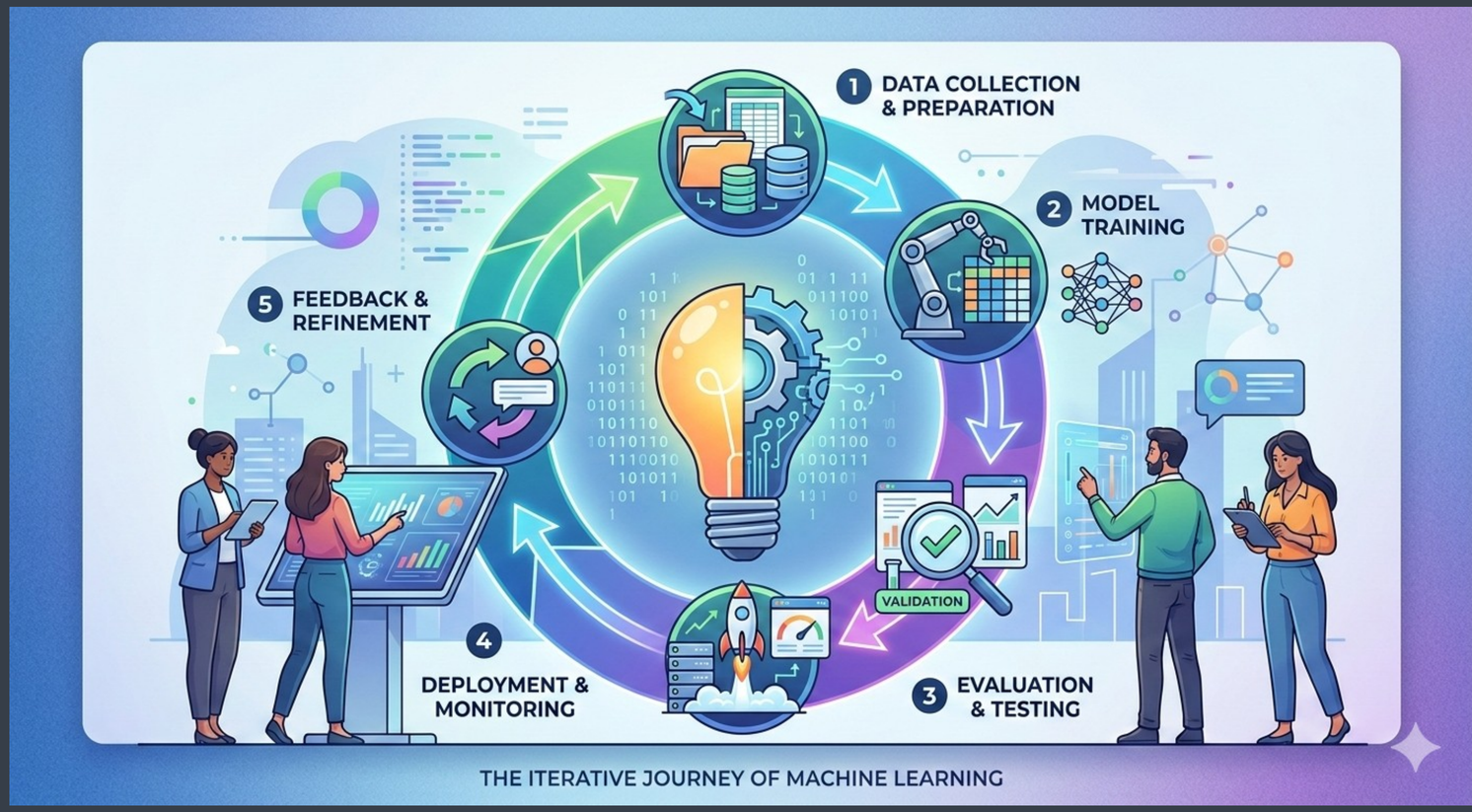


Feature Pipelines

"One-and-Done" Fallacy: If you clean your training data manually, how will you clean the live data arriving at 2:00 AM?

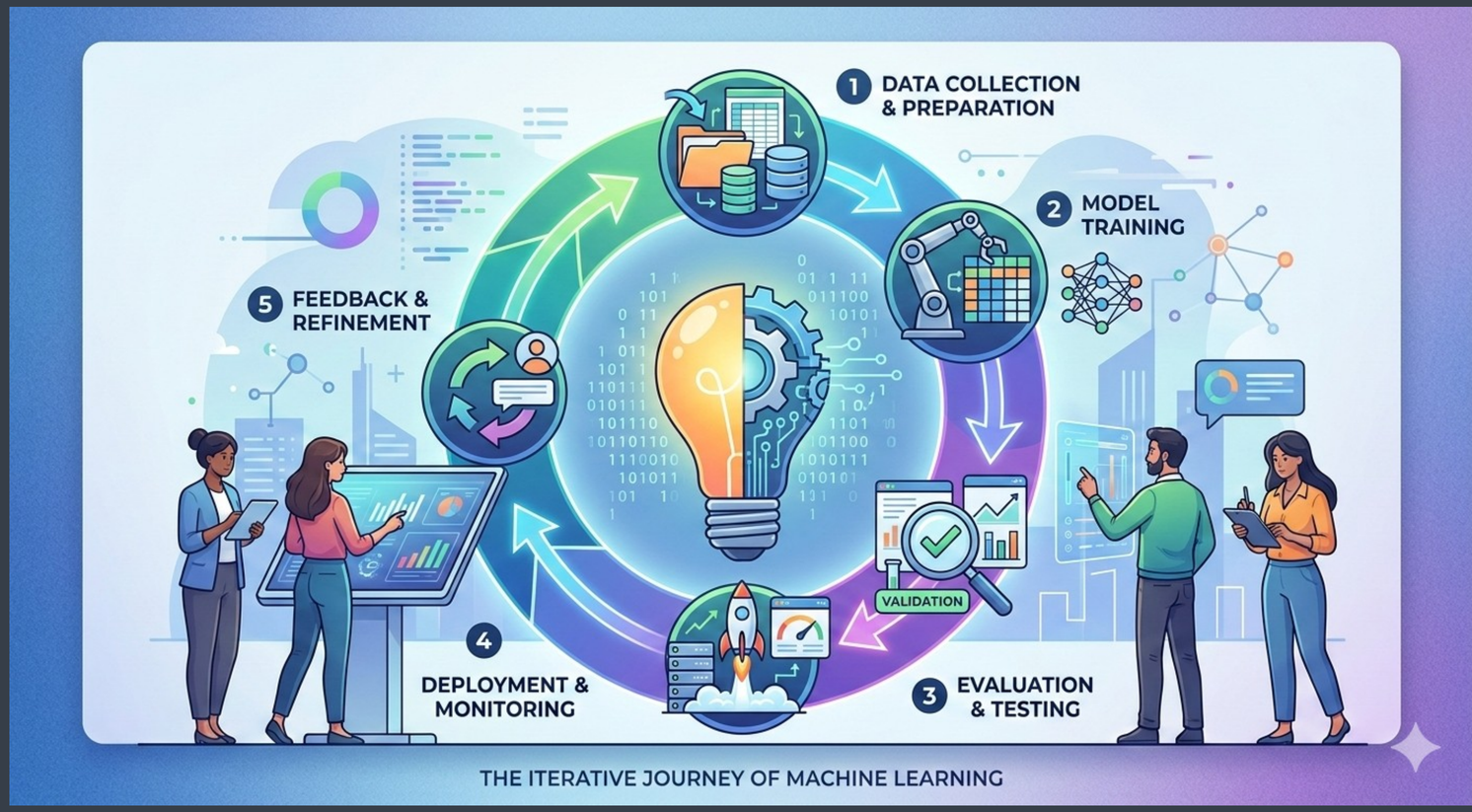
Feature Pipeline Definition: An automated, repeatable sequence of transformations that turns "Raw Data" into "Model-Ready Input."

We don't "fix" data; we write **code that fixes data**. (**Most time consuming task for beginners in a project**).



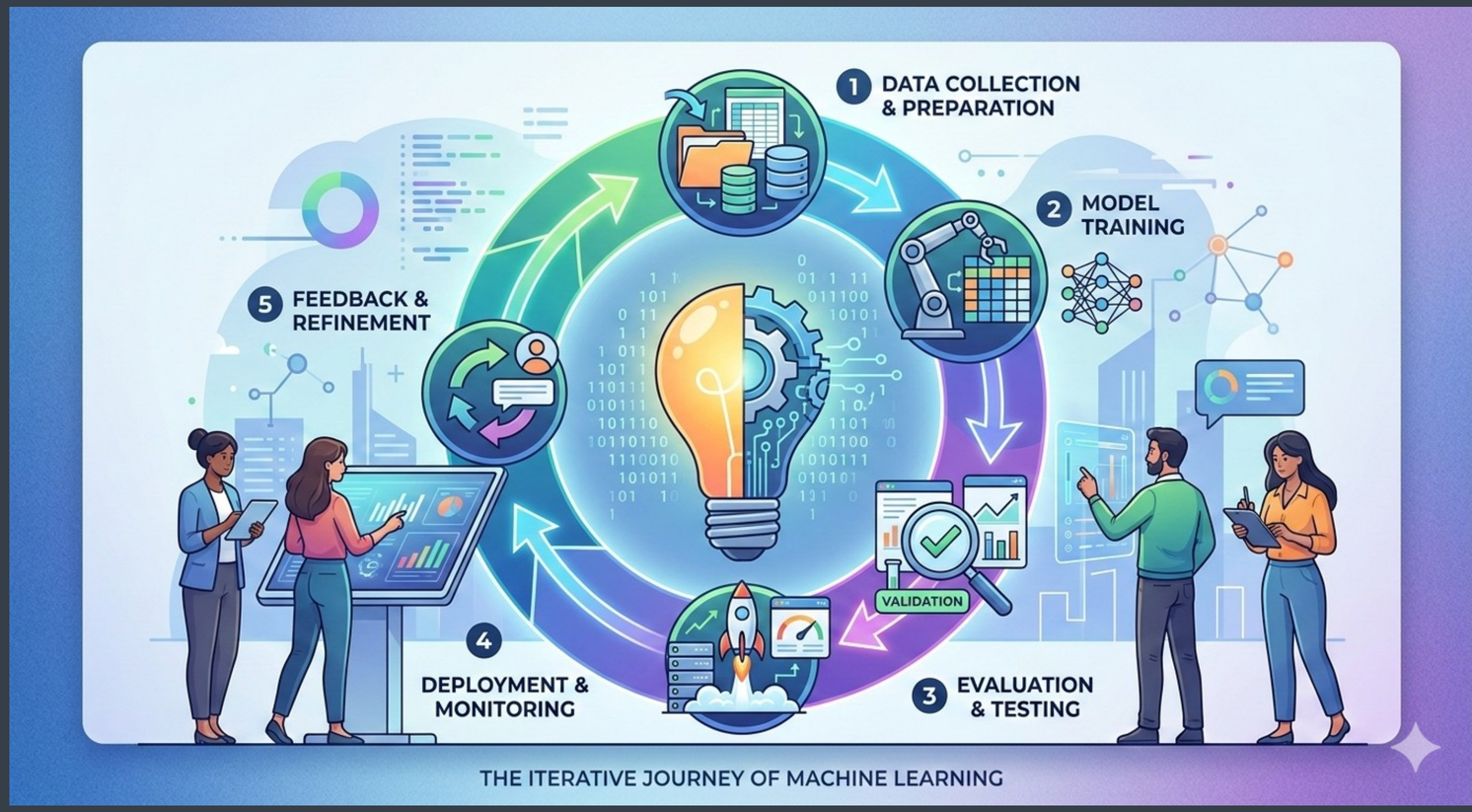
Feature Pipelines

Raw data is rarely the best predictor; the "derived" feature is where the value lies.
- Transformation example: Converting a "Timestamp" into "Is it a Weekend?" or "Is it High-Trading Volume hours?"
- Aggregation: Turning a list of 100 transactions into "Average Spend per Hour."
- Domain Expertise: Why a technologist/economist is better at feature engineering than a generic algorithm?



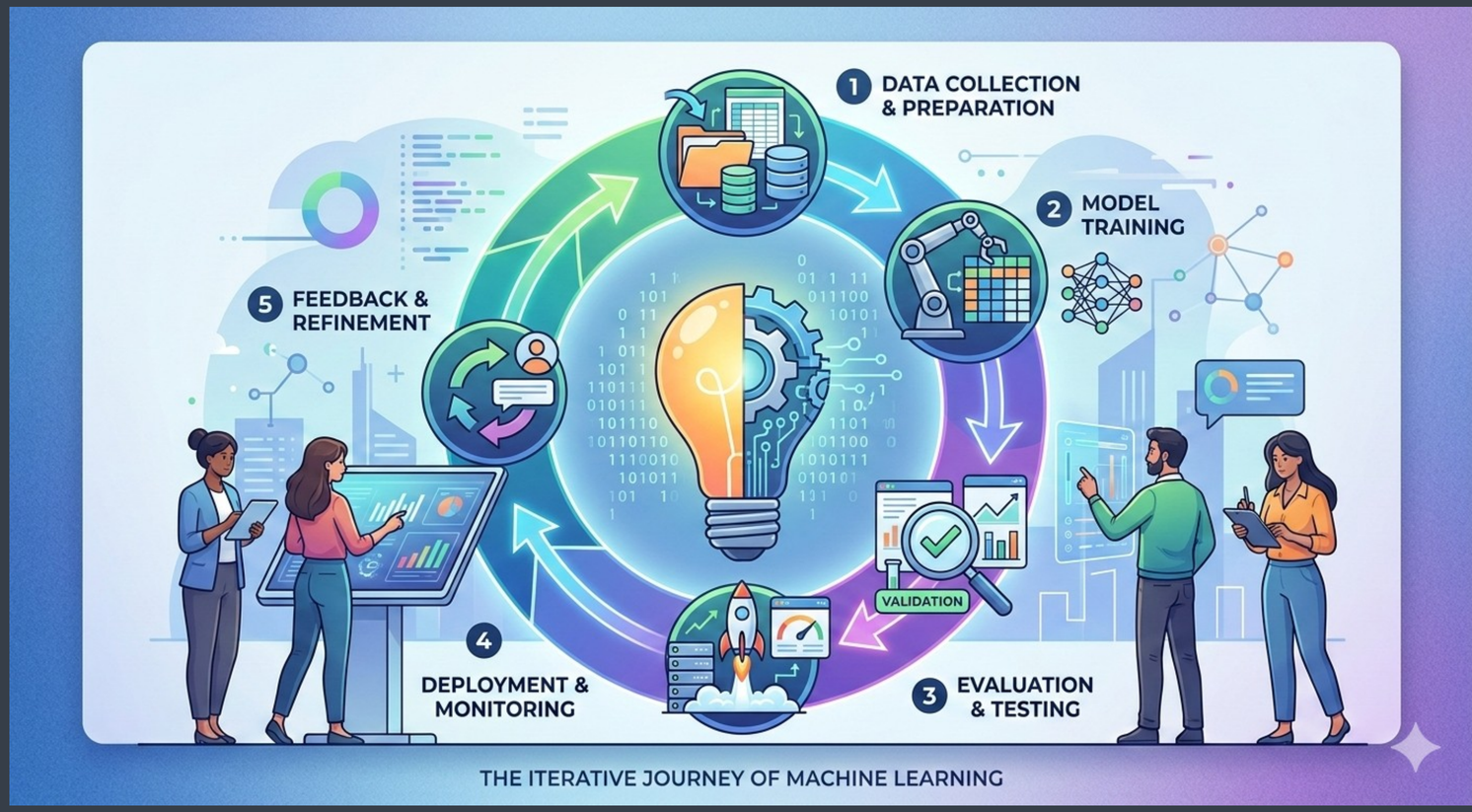
Feature Pipelines

Training-Serving Skew is major issue in production models.
- When the math used to calculate a feature during training differs from the math used in the live app.
- If you normalized data using the "Average" of the 2024 dataset, but the live app uses the "Average" of the 2025 dataset, the model’s input is skewed.
- The exact same pipeline code must run in both the "Lab" and the "Field."



Feature Pipelines

An Idea: Decoupling feature creation from model training.
- Recalculating the same features for different models is a waste of compute.
- **Feature datasets and a Feature Store of these datasets** acts as a single source of truth.
- Point-in-Time Correctness: Ensuring that when you train, you only see the features as they existed at that specific moment in the past.



The Reproducibility Crisis

Lightweight Feature Store Options

DIY (Composable Stack):
- Storage: PostgreSQL / Amazon S3, Processing: Pandas / Apache Spark, Versioning: Git
- Low cost, flexible. Manual governance. Weak lineage and online/offline consistency

Embedded Feature Management in Pipelines:
- Tools like dbt and Airflow.
- Features = curated tables/views. Materialized in warehouse (Snowflake, BigQuery, etc.)
- Good for batch ML, but not good for real-time applications.

The Reproducibility Crisis

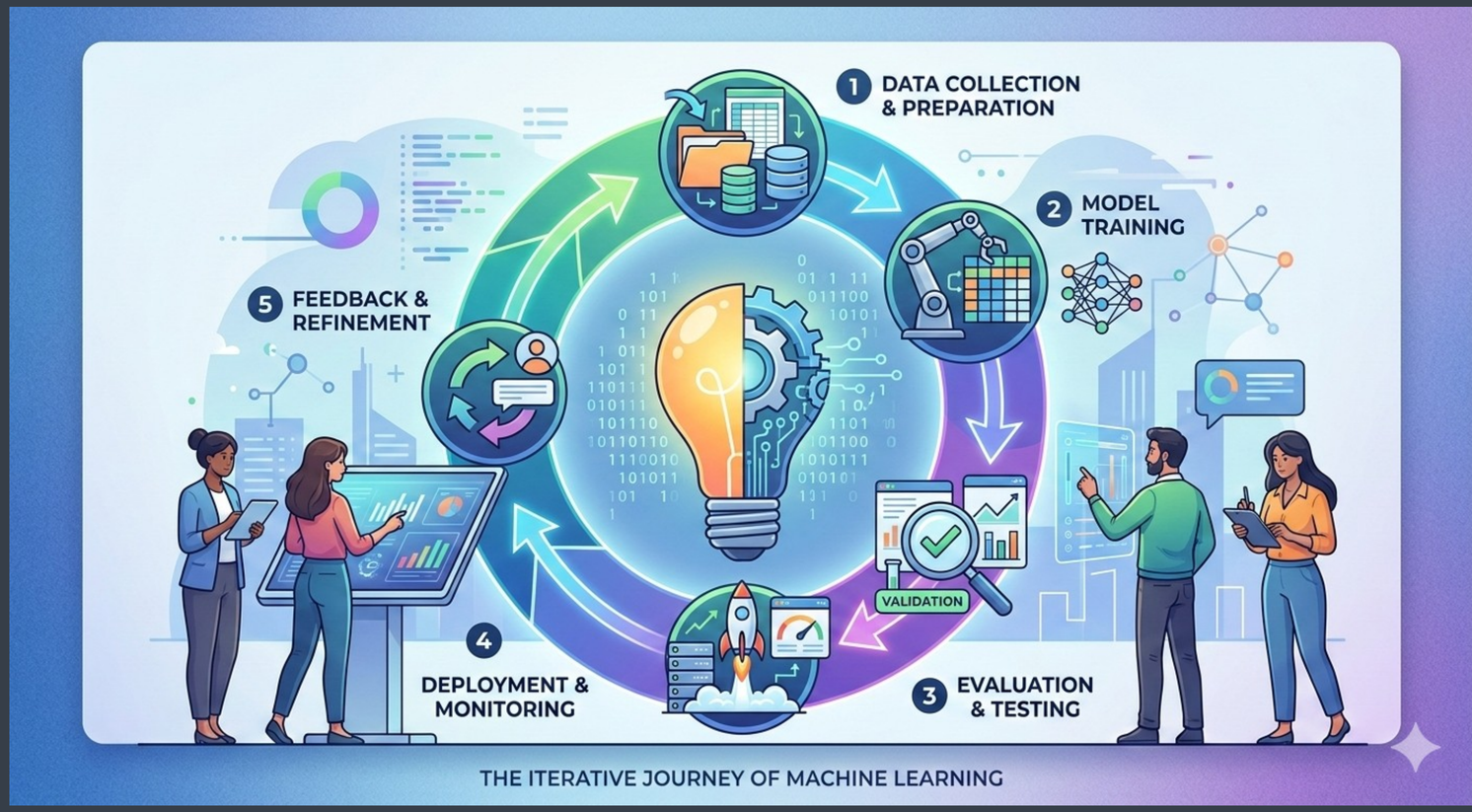
Advanced Feature Store Options

Fully Managed Platforms (Cloud-Native):
- Amazon SageMaker Feature Store, Google Vertex AI Feature Store
- Native integration with ML pipelines, Online + offline sync, Auto-scaling, security, monitoring

Enterprise MLOps Platforms (On-Premise, Regulated Environments):
- Tecton, Hopsworks
- Real-time feature pipelines (streaming support), Advanced governance and access control, Built-in monitoring and drift detection

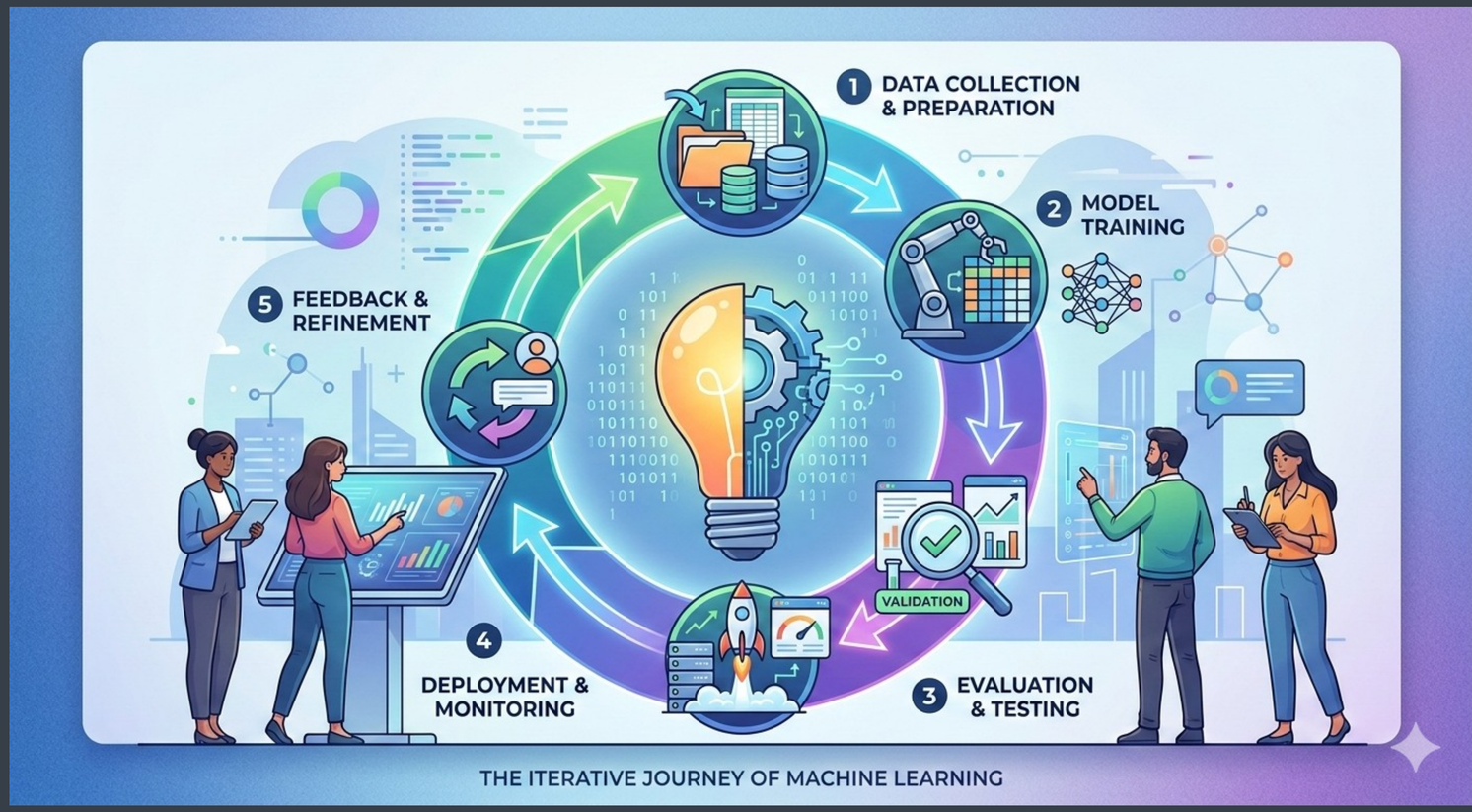
Feature Pipelines

Automation & Monitoring (The Health Check)
A model won't crash when a feature pipeline breaks; it will just give **confidently wrong answers**.
- Data Quality Gates: If a feature that is usually between 0 and 1 suddenly becomes 100, the pipeline should stop the model.
- Null Handling: What does the pipeline do when a sensor fails? (Imputation vs. Dropping).



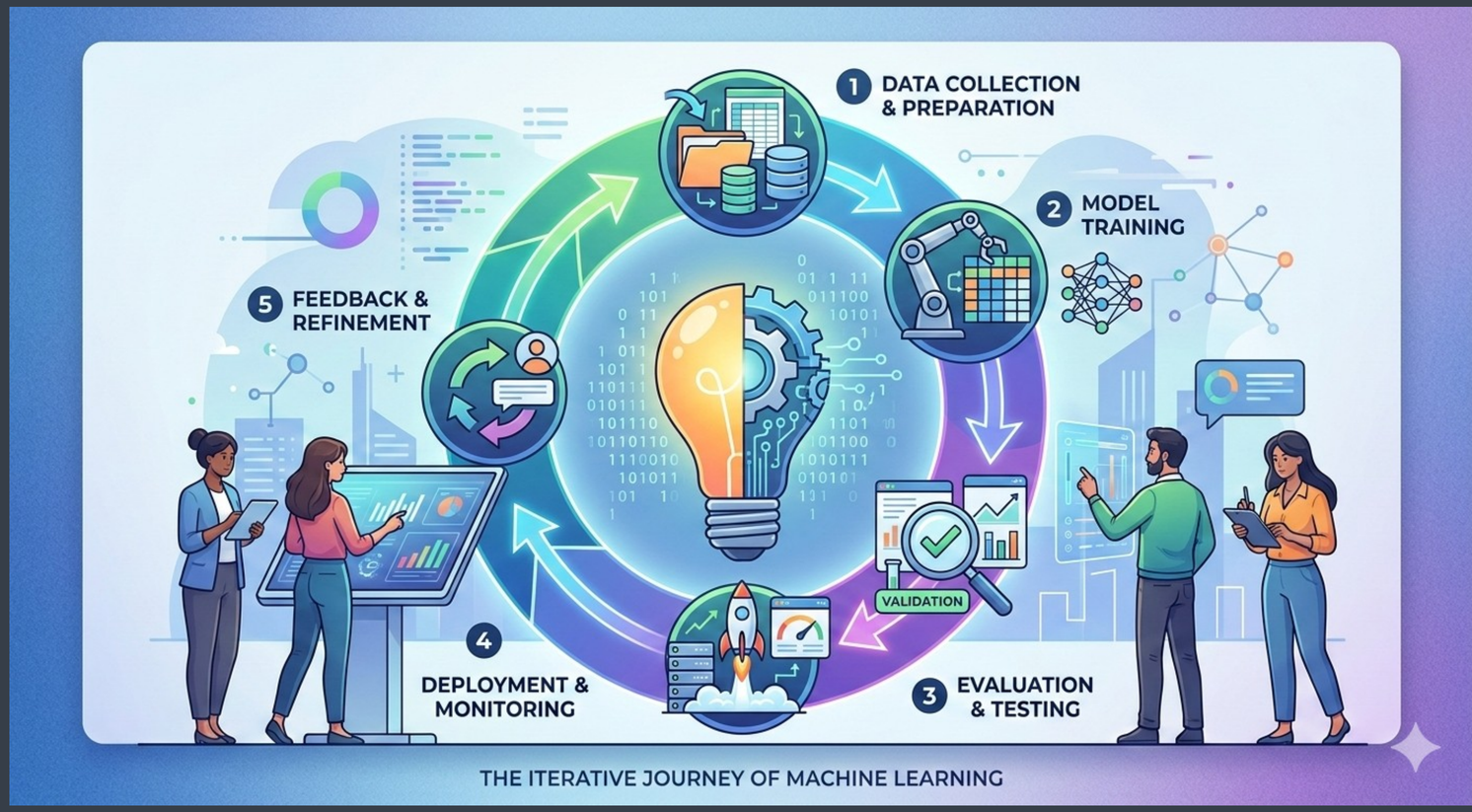
Failure Modes

Unlike traditional software, ML models don't "crash"—they just become wrong.
- Degradation vs. Bugs: A database error throws an exception; an ML model simply provides a high-confidence incorrect prediction.
- The Trust Gap: How a model that worked perfectly for six months can suddenly start losing money without a single line of code changing.
- The Dashboard Illusion: Why looking at "Average Accuracy" can hide localized disasters.



Failure Modes

Data Drift: The distribution of the data you are seeing now is not what you trained on.
- Sensor/System Change: A website update changes how a "click" is recorded, or a new hardware sensor has different calibration (Ref: FPGA/Hardware optimizations).
- Seasonal Shifts: A model trained on winter retail data fails in July.
- Detection: Use statistical tests to compare "Training Data Distribution" vs. "Live Data Distribution."



The Reproducibility Crisis

Concept Drift (The World changed): The input data is the same, but the meaning of the data has changed.

- Imagine you are a detective who has learned a rule: "If a person is wearing a suit and carrying a briefcase, they are going to work." For decades, this rule (the "concept") holds true.
- Then, a pandemic hits. Now, a person wearing a suit and carrying a briefcase might just be heading to a Zoom call in their living room, or perhaps they are dressing up to feel normal during a lockdown.
- The inputs (Suit + Briefcase) are identical. But the meaning (Going to the office) has evaporated. The concept has drifted.

The Reproducibility Crisis

Machine learning models are essentially just complex mathematical equations trying to find a relationship: y = f(x).

Concept Drift means the function f has changed, even if x looks normal.
Example: A bank has a fraud detection model.
- Old Reality (f): Large transactions at 2:00 AM are highly suspicious (fraud).
- New Reality: The bank serves a 24/7 gaming platform. Suddenly, large transactions at 2:00 AM are normal (gamers buying loot boxes).

If the model isn't updated, it will flag thousands of legitimate gamers as fraudsters (false positives) and potentially miss new types of fraud that happen during the day.

The Reproducibility Crisis

Models are trained on historical data, which is essentially a "snapshot" of the world at a specific time. The world, however, does not stay still.

Examples
- Post-COVID Consumer Behavior: Before 2020, a spike in purchasing "toilet paper and hand sanitizer" was an anomaly. After March 2020, it became a standard weekly shop. A model trained on 2019 data would treat normal 2020 behavior as a weird anomaly or a bulk-buying bot attack.
- Market Volatility (HFT): In High-Frequency Trading, relationships between commodities can last seconds or minutes. If a war breaks out in an oil-producing region, the correlation between "Oil Price" and "Airline Stock Price" might instantly reverse. A model relying on the previous correlation will lose money instantly.

The Reproducibility Crisis

How fast the world changes dictates how you must react.

Sudden Drift (The Black Swan)

- What it is: An instantaneous, radical shift.
- Example: A new law passes banning a specific chemical. Suddenly, products containing that chemical go from "high sales" to "zero sales" overnight.
- Impact: The model breaks immediately. It requires instant intervention or a shutdown switch.

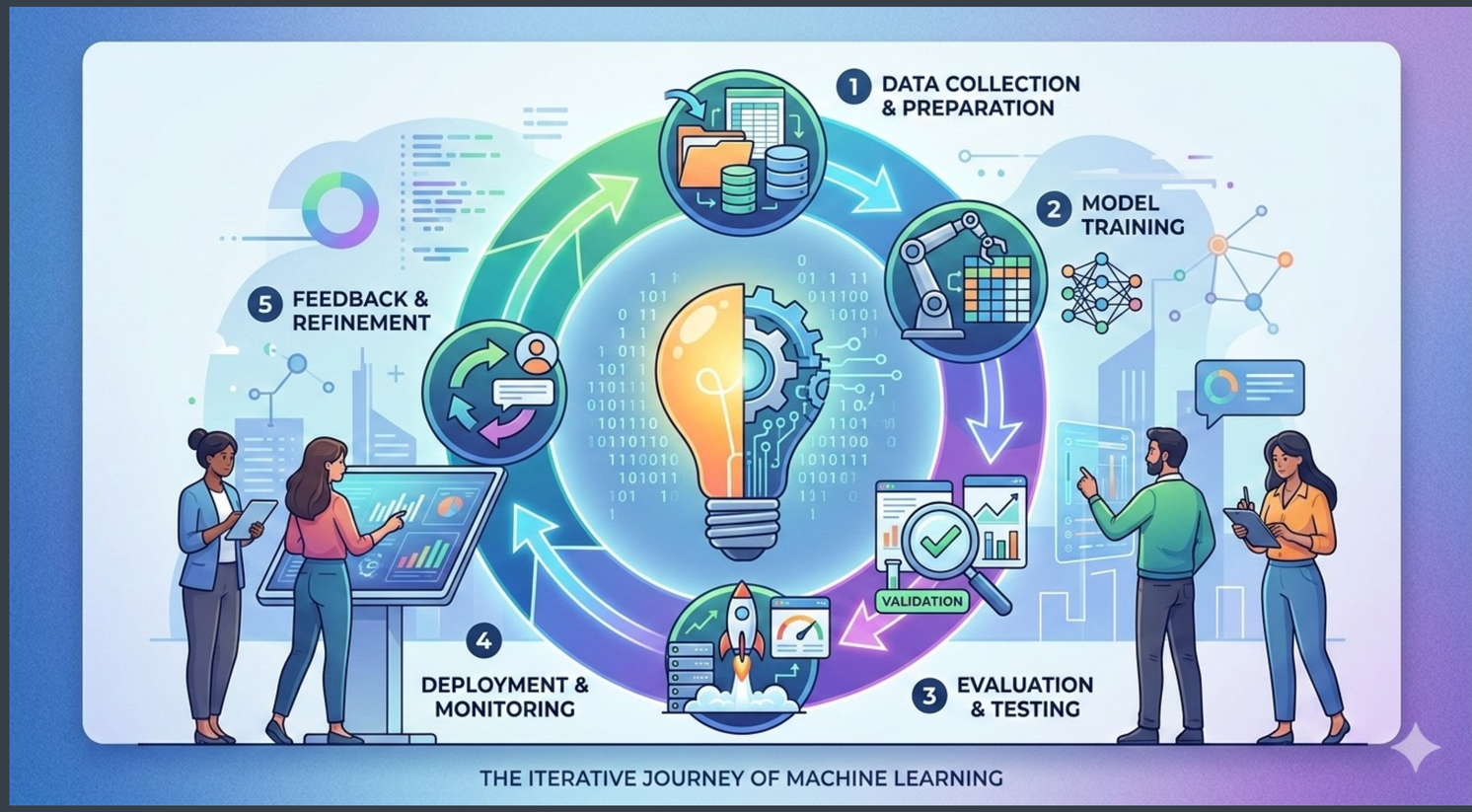
Gradual Drift (The Boiling Frog)

- What it is: A slow, steady evolution of behavior that is hard to notice day-to-day.
- Example: Fashion trends. A model predicting "What is trendy?" sees "Skinny Jeans" slowly lose popularity to "Baggy Jeans" over two years.
- Impact: The model degrades slowly. Accuracy drops by 0.1% per month. It is often ignored until it becomes a crisis, but because it happens slowly, it is often mistaken for "noise" rather than a fundamental shift.

Failure Modes

Never trust a new model (the Challenger) until it proves itself against the current one (the Champion).

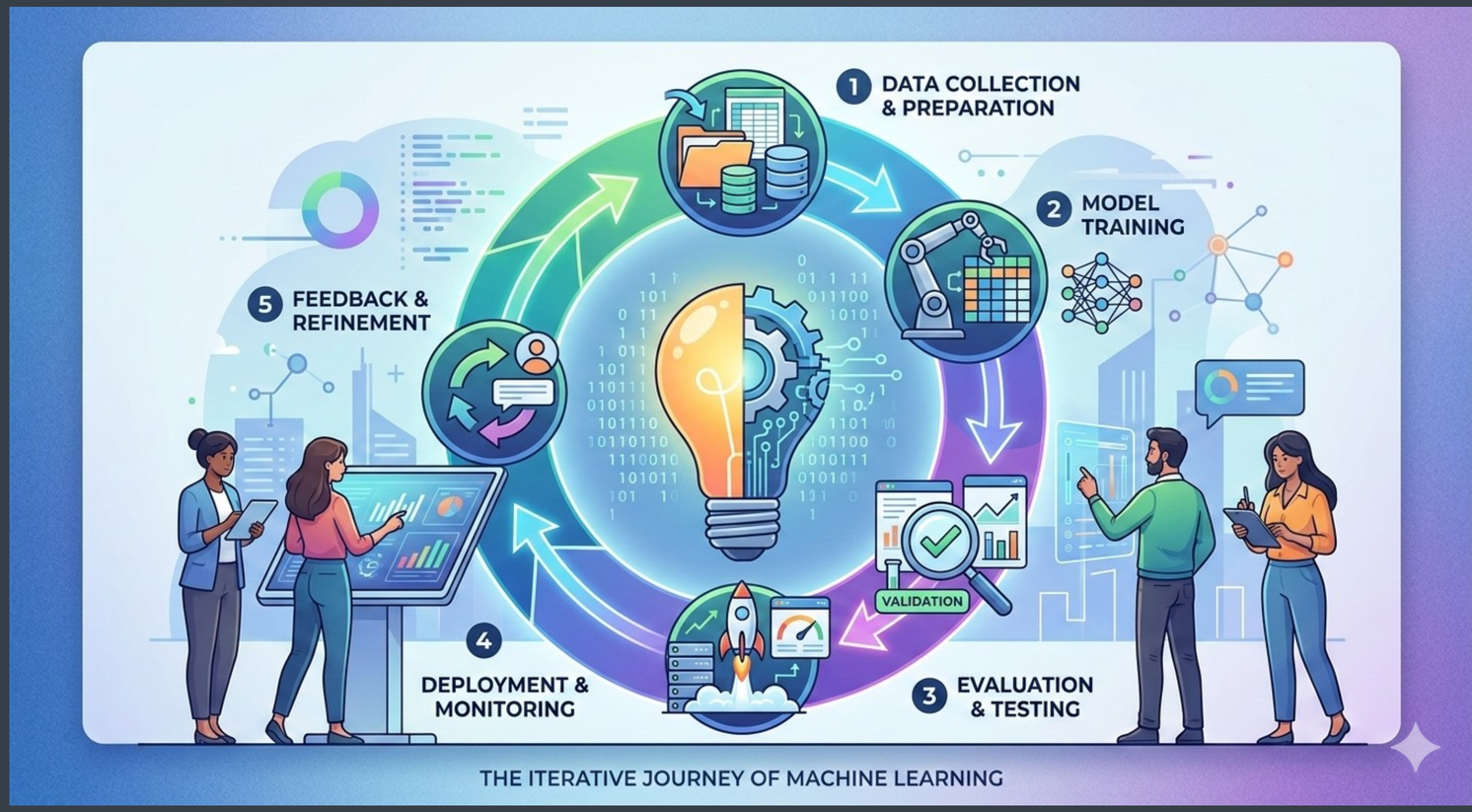
- Shadow Deployment: Running the new model in the background, seeing its predictions, but not letting it "drive" yet.
- A/B Testing in ML: Comparing the business outcomes of two different models in real-time.
- The "Kill Switch": Knowing when to automatically revert to a simpler, rule-based heuristic if both models fail.



Failure Modes

The Feedback Delay Trap: The danger of "Flying Blind" when you don't know the truth immediately.

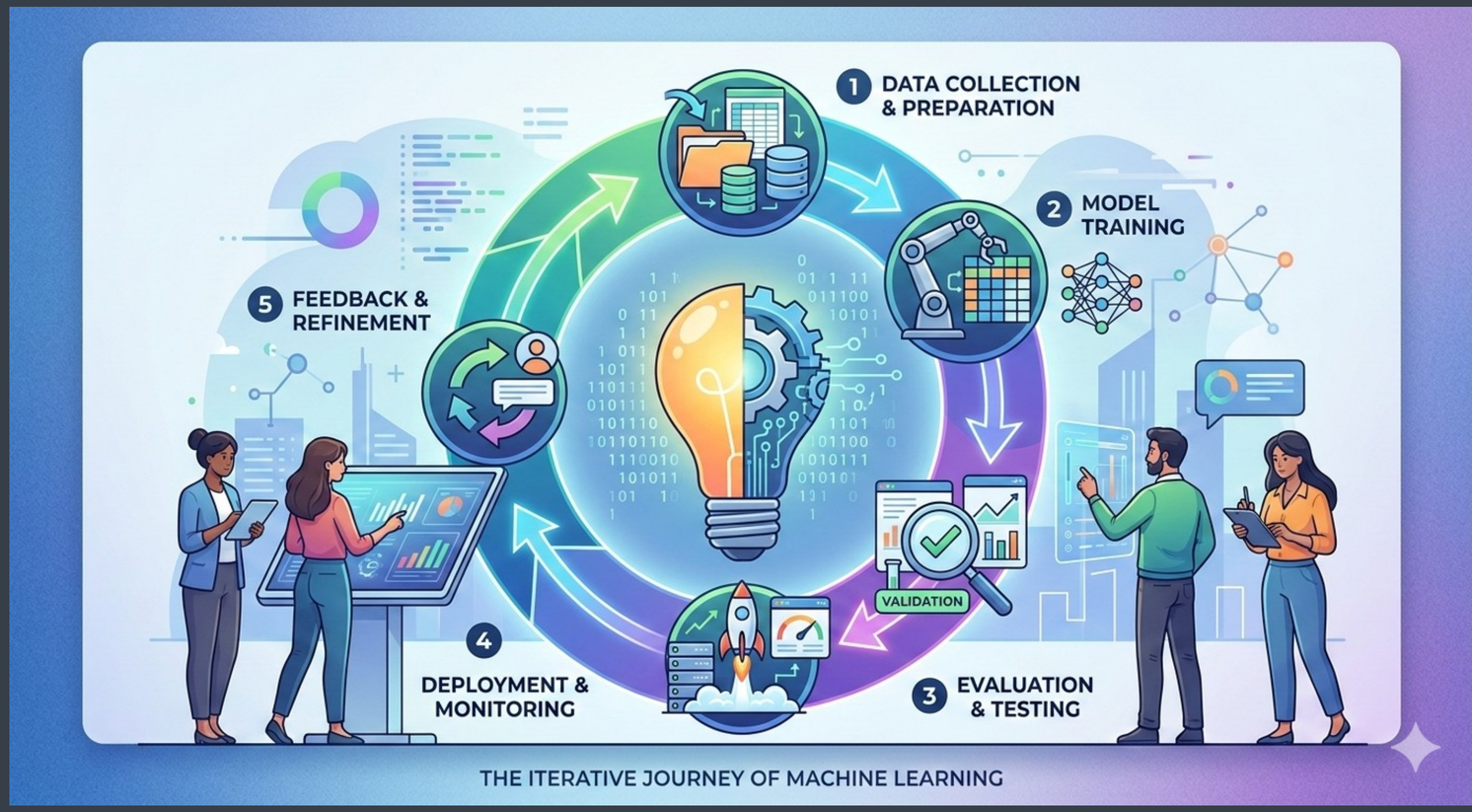
- Immediate vs. Delayed Labels: In ad-clicks, you know the result in seconds. In loan defaults or logistics pathing, you might not know if the model was "right" for months.
- **Proxy Metrics**: What to monitor when you don't have the "Ground Truth" yet.
- The Opportunity Cost: The price of waiting for perfect data versus the risk of acting on a decaying model.



Taxonomy of Algorithms

A high-level view of Supervised vs. Unsupervised learning based on the availability of "Ground Truth."

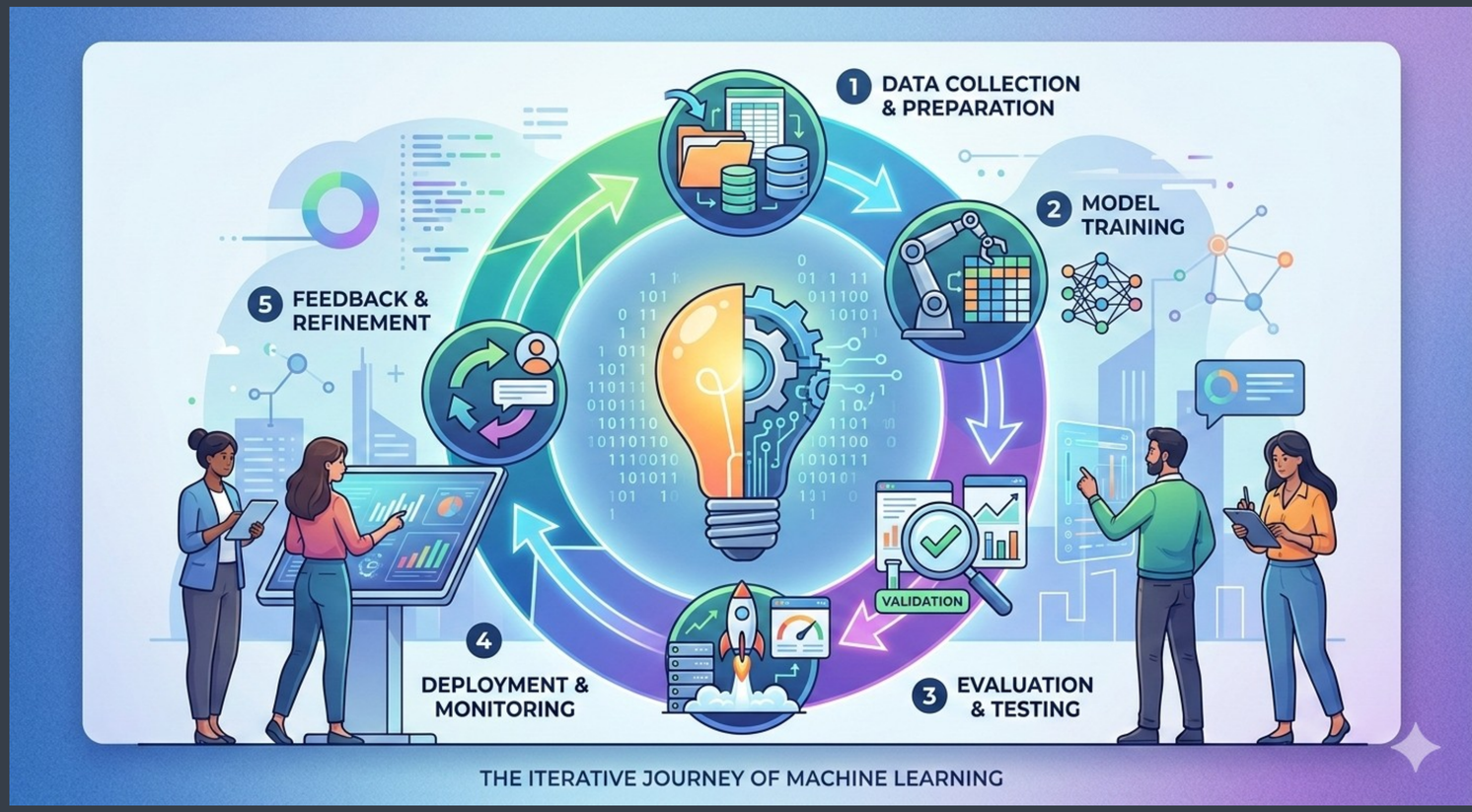
- Supervised: Learning from a "labeled" past (Input + Answer). You have a teacher (the historical data) telling the model what is right.
- Unsupervised: Finding hidden patterns in "unlabeled" data. There is no "right" answer, only clusters and anomalies.
- The Transition: Moving from "What happened?" to "What belongs together?"



Taxonomy of Algorithms

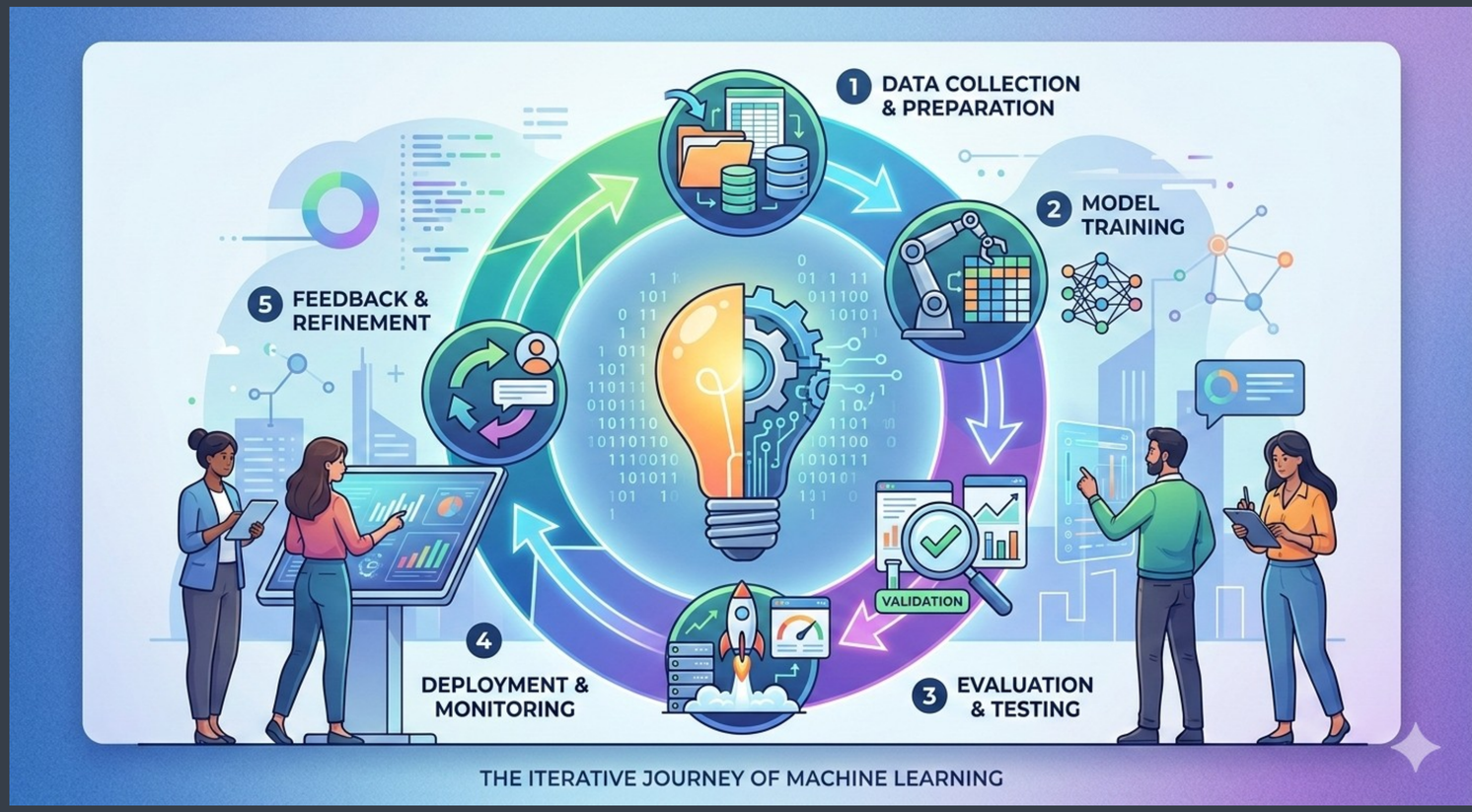
Predictive Utility (Regression): Estimating continuous values to drive planning and pricing.

- Business Question: "How much?" or "When?"
- Use Cases: Demand forecasting for logistics, pricing a house, or predicting the time to failure for a machine.
- Beyond SPSS: It’s not about the p-value; it’s about the Mean Absolute Error (MAE)—how many dollars/minutes are we actually off?



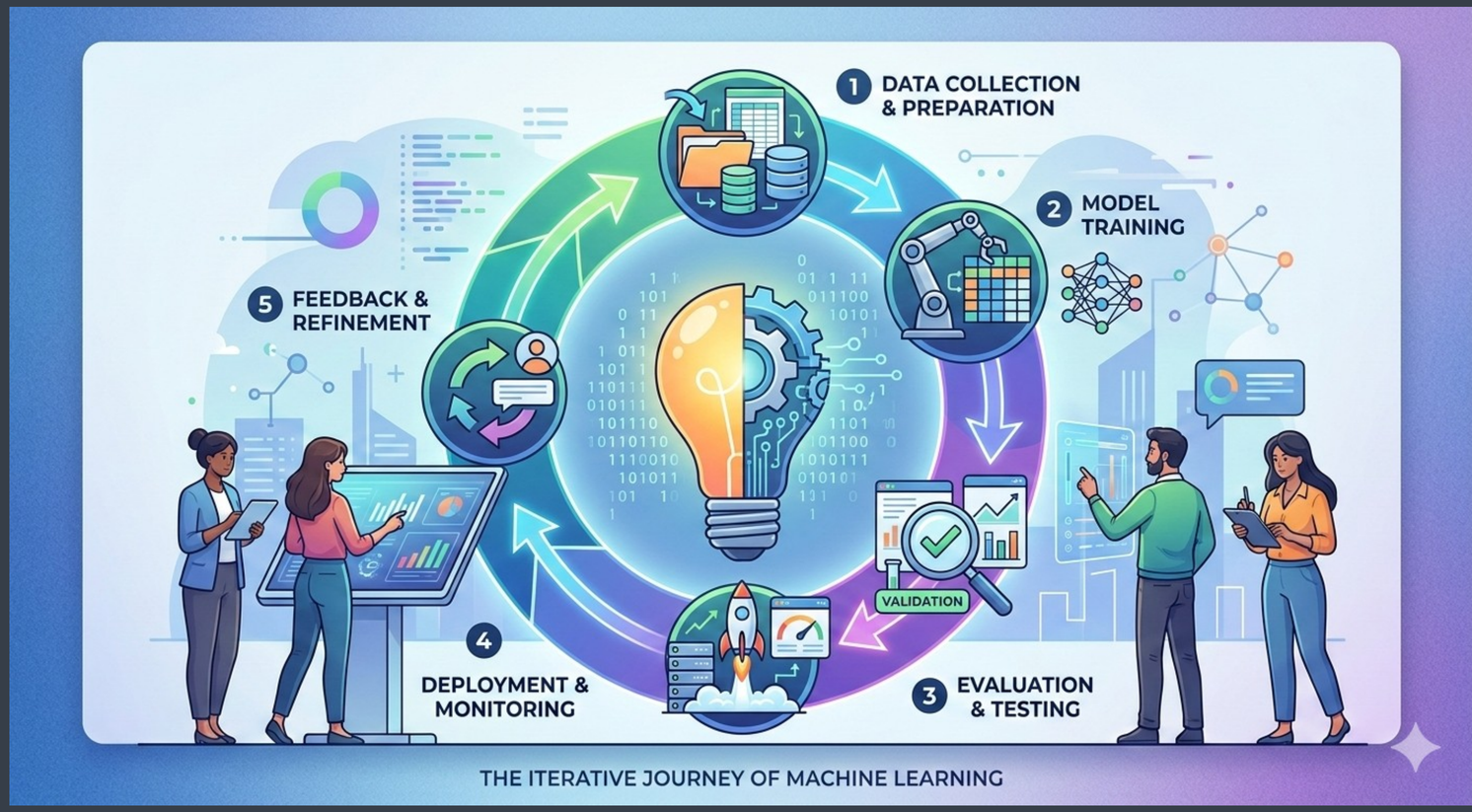
Taxonomy of Algorithms

Decisive Utility (Classification): Sorting entities into buckets to automate high-volume decisions.
- Business Question: "Which one?" or "Is this X?"
- Use Cases: Churn prediction (Will the customer leave?), Fraud detection (Is this transaction stolen?), or Lead scoring (Is this a 'Hot' lead?).
- The Utility Gap: Discussing why a "False Negative" (missing a fraud) is much more expensive than a "False Positive" (blocking a real user).



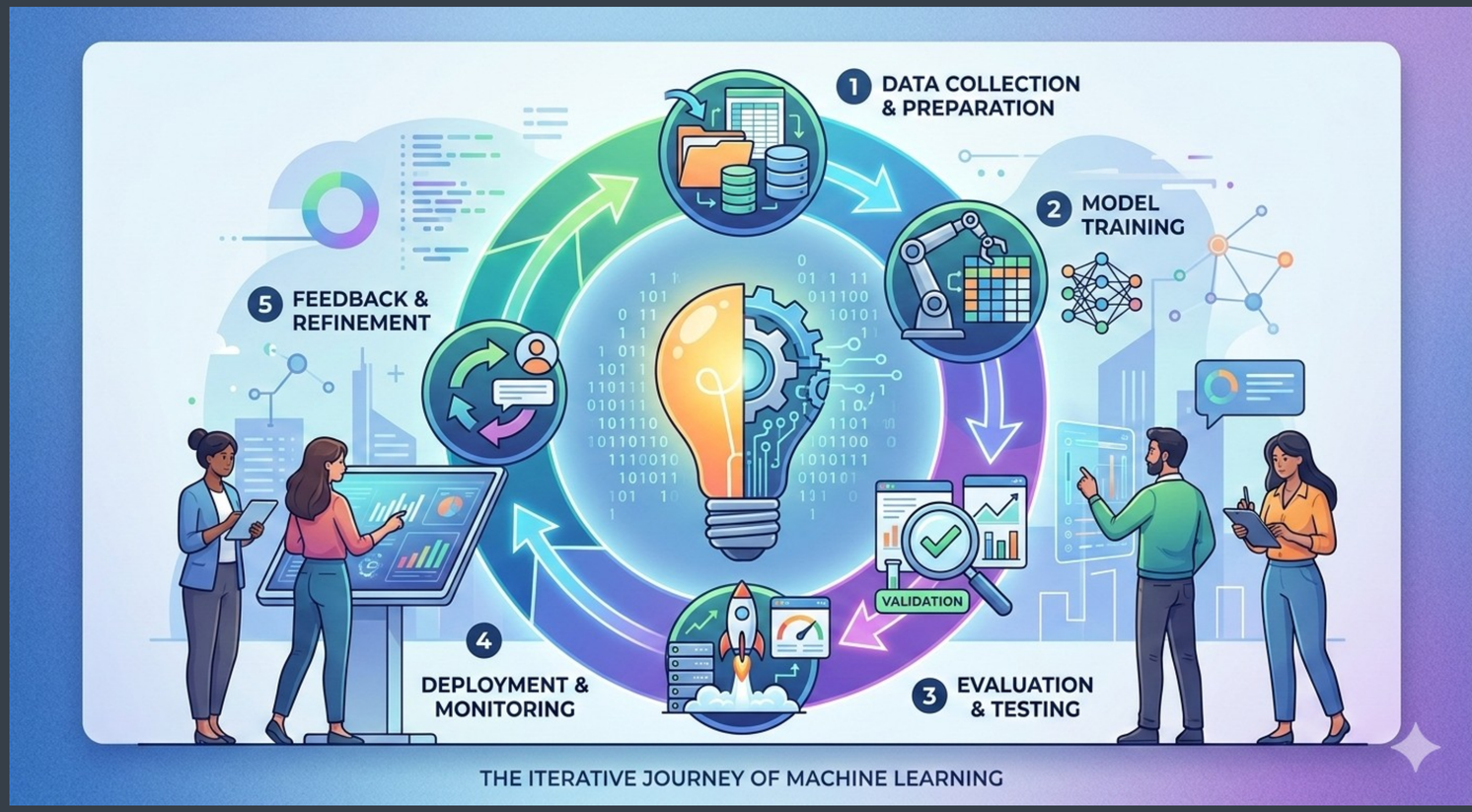
Taxonomy of Algorithms

Grouping Utility (Clustering & Association): Discovering market segments and relationships that humans miss.
- Business Question: "Who are my different types of customers?"
- Use Cases: Customer segmentation (Persona building), Market Basket Analysis (People who buy X also buy Y).
- Strategic Value: Using these insights to design different marketing strategies or store layouts without having a predefined "correct" category.

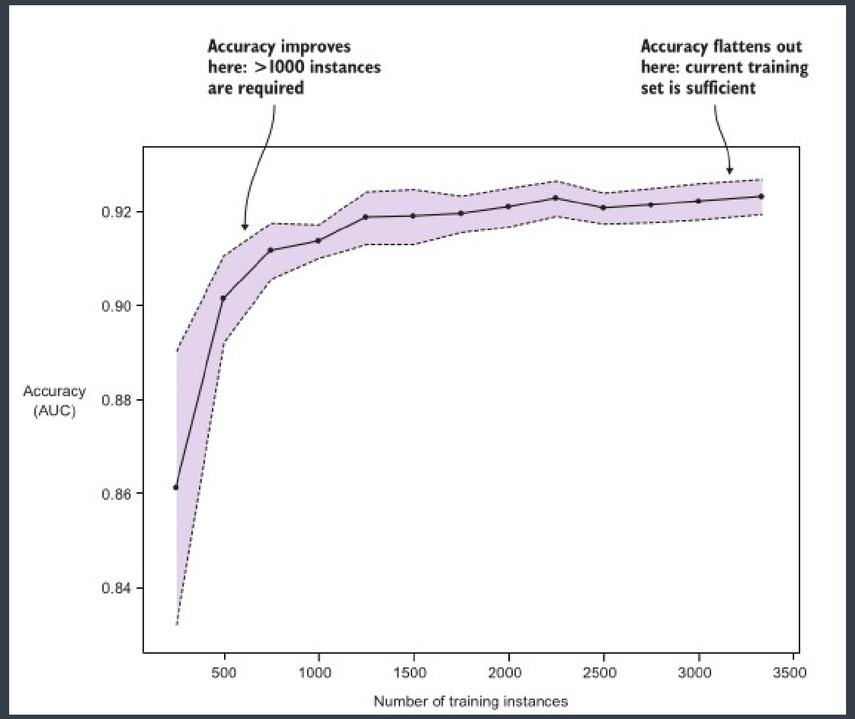


Taxonomy of Algorithms

Right tool for the right job: Choose an algorithm based on business constraints, not just accuracy.
- Interpretability vs. Performance: Do you need to explain why a decision was made (e.g., a loan rejection) or do you just need the highest speed (e.g., HFT)?
- Data Volume: Deep Learning (needs millions of rows) vs. Tree-based models (great for smaller, structured tables).
- Operational Cost: The cost of running the "best" model vs. the "good enough" model.



Taxonomy of Algorithms

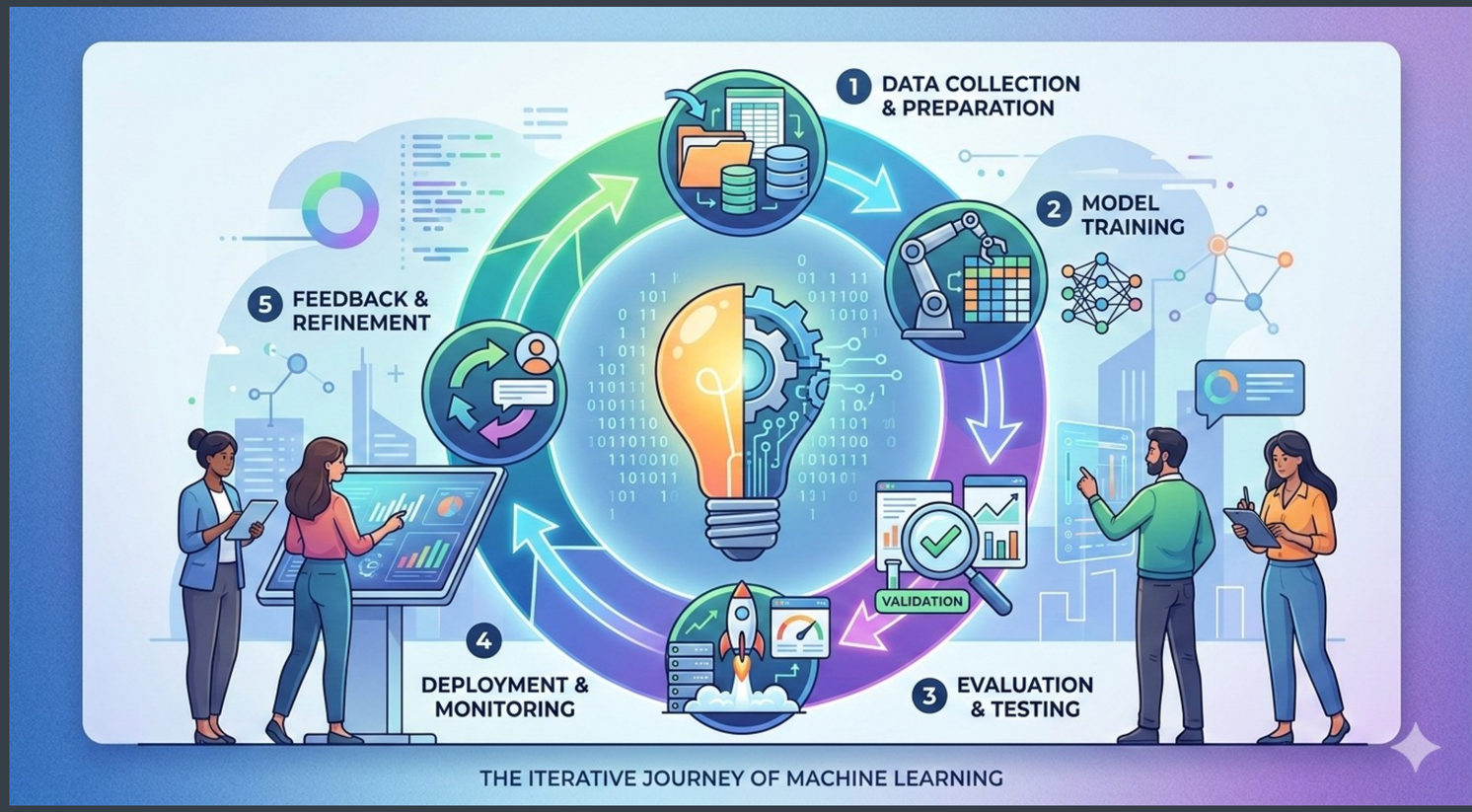


Notes on Data Visualization

Between data collection/preprocessing and ML model building lies the important step of data visualization.

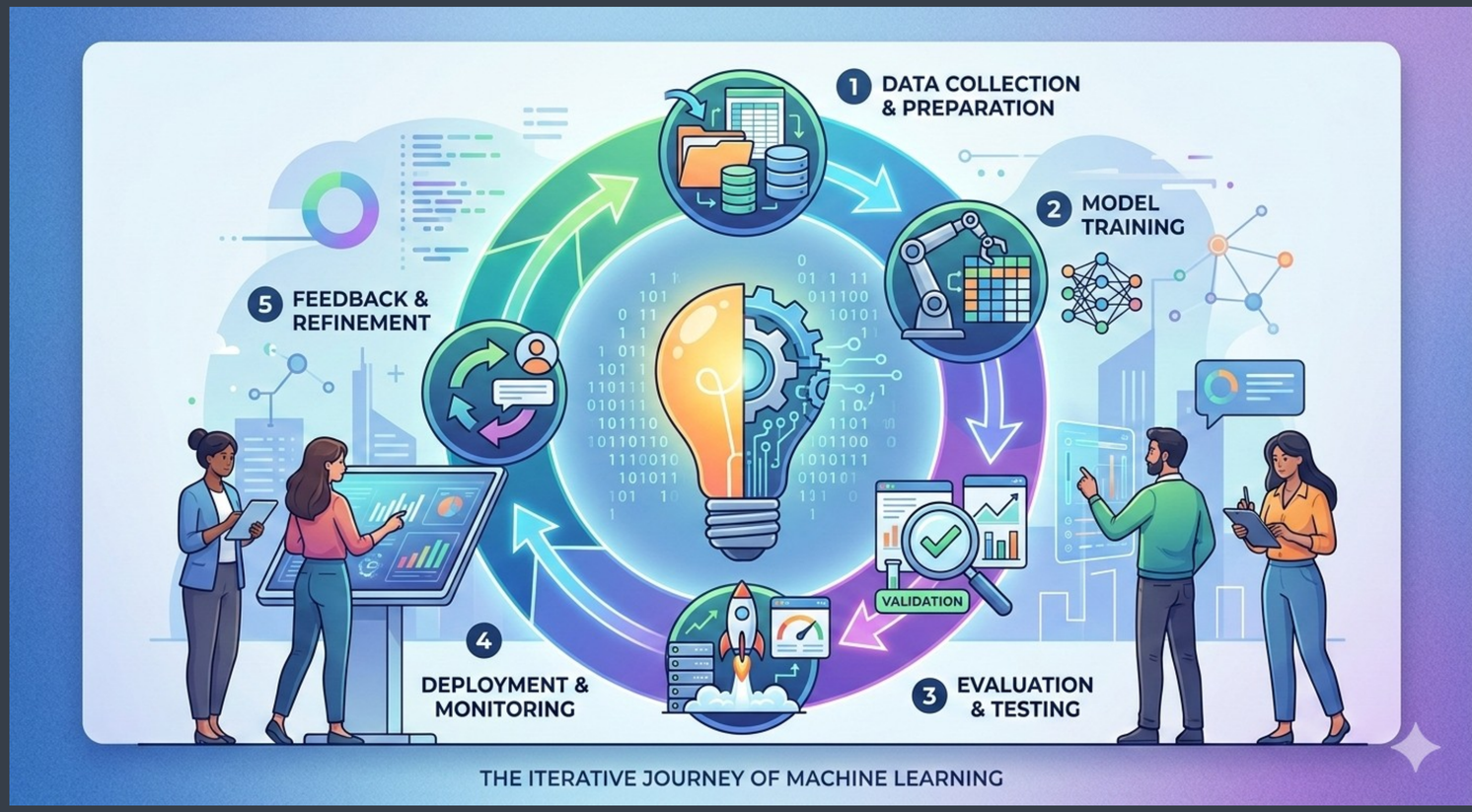
Data visualization serves as a sanity check of the training features and target variable before diving into the mechanics of machine learning and prediction.

Based on number of variables/ features you may need multiple plots.

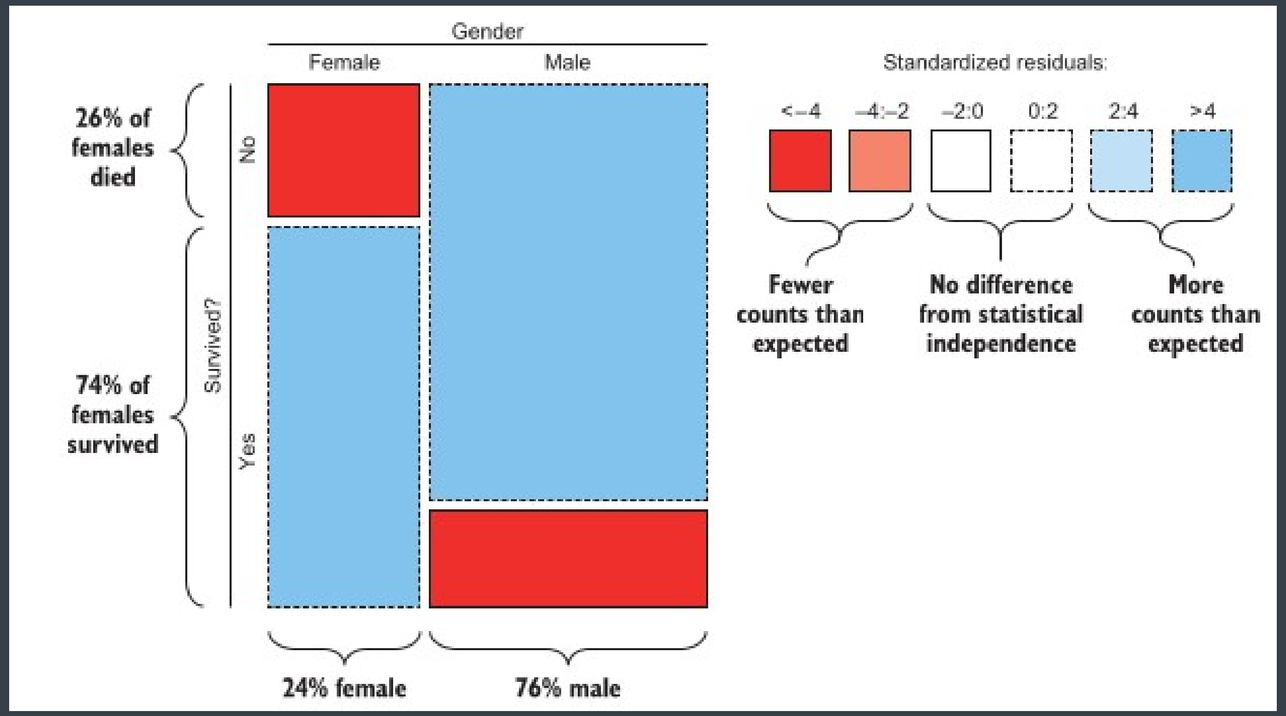


Notes on Data Visualization

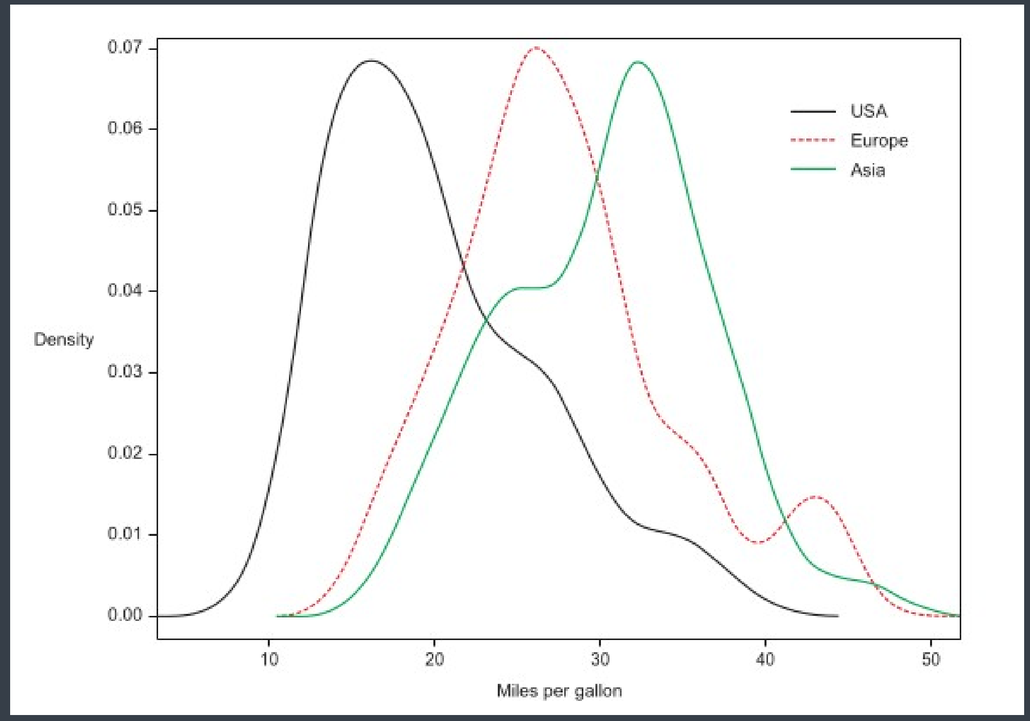
Four common plot types to use in pre-analysis visualization
- For categorical input and output: Mosaic plots
- For categorical input but numeric output: Density plots
- For numerical input and output: Scatter plots
For numerical input but categorical output: Box plots.



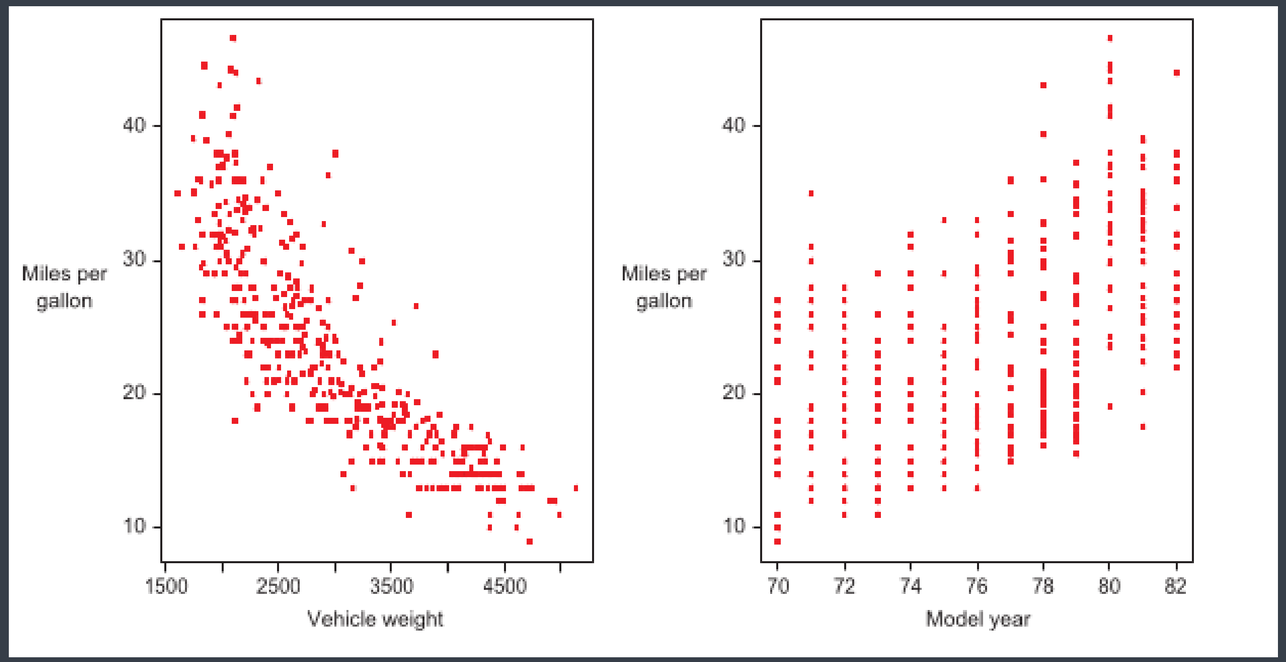
Notes on Data Visualization



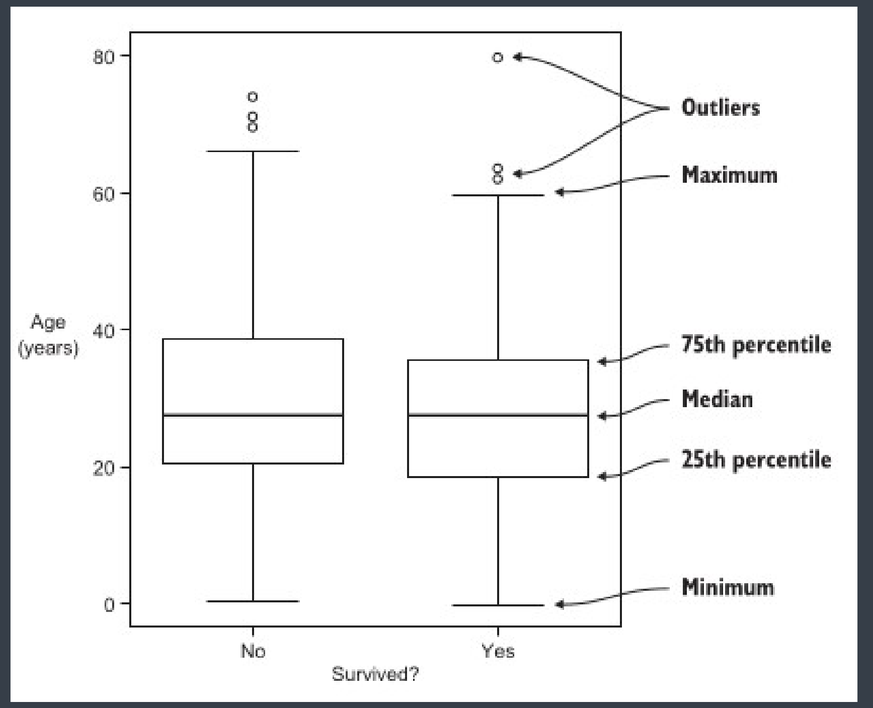
Notes on Data Visualization



Notes on Data Visualization

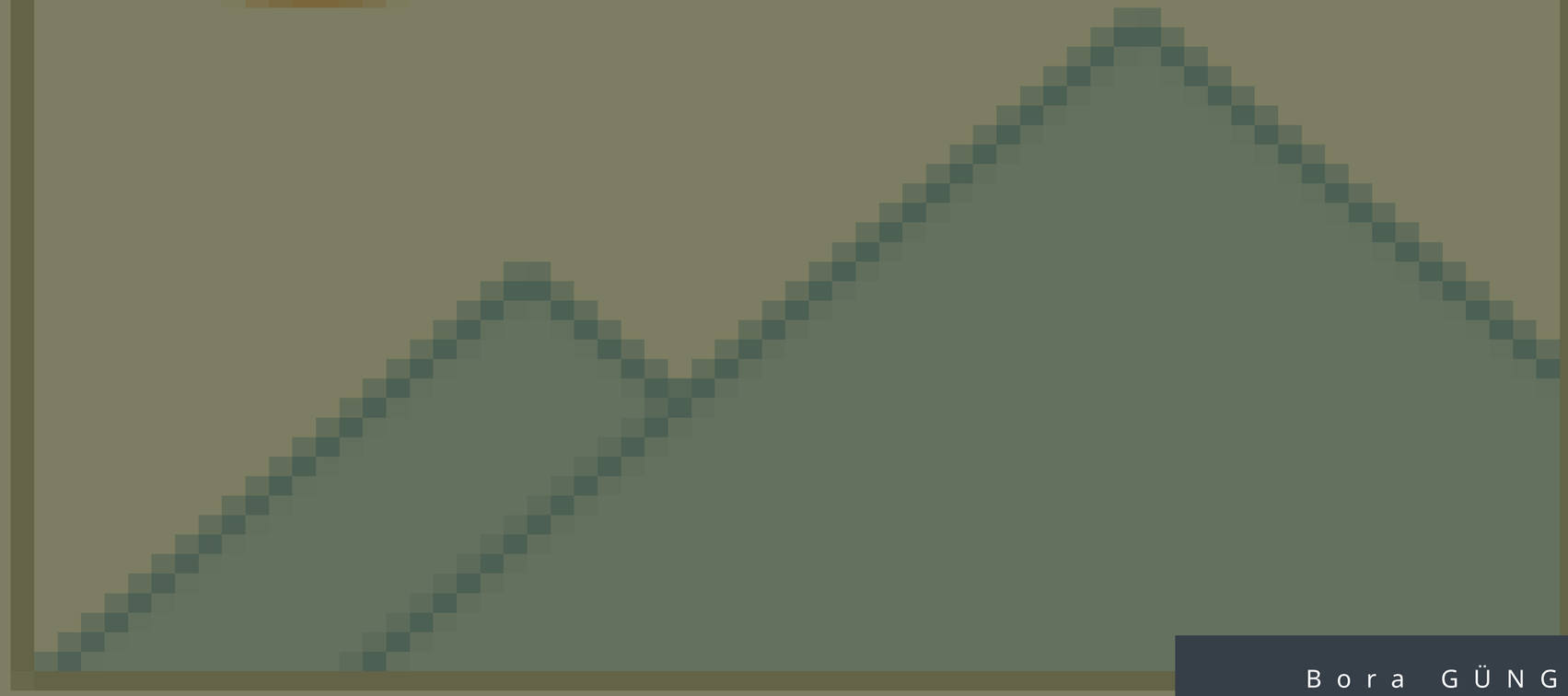


Notes on Data Visualization



Thank You

Please follow up on Github and your emails (school emails).



Bora GÜNGÖREN